



SOL

Programming Language

Language and Toolchain Design Specification

Concept Design v0.2

September 9, 2026

Implementation baseline 6ec4ba9 / through P2.6

A LANGUAGE OPTIMIZED FOR HUMAN-AI CO-DEVELOPMENT

STATUS

Concept Draft with Executable-Core Decisions

Target design with an executable reference interpreter and experimental production internals. E1-E6, P1, and P2.1-P2.6 complete; P2 open. Not production-ready.

Document Control

Field	Value
Document	Sol Programming Language - Language and Toolchain Design Specification
Version	0.2 Concept Draft; September 2026 documentation revision
Date	September 9, 2026
Status	Concept Draft with Executable-Core Decisions
Baseline	Implementation 6ec4ba9; E1-E6, P1, P2.1-P2.6 complete; P2 remains open.
Primary objective	Define a language whose semantics, tooling, and source representation optimize safe maintenance by humans and AI systems.
Normative vocabulary	MUST, MUST NOT, SHOULD, SHOULD NOT, and MAY distinguish required, recommended, and optional behavior.
Authority	docs/specification.typ is the editable manual source. The root v0.2 PDF is generated; TODO.md alone owns live status and order.

SCOPE This document records the target design and bounded implemented decisions. Unless explicitly marked **IMPLEMENTED** or described as current bootstrap behavior, passages and examples are proposals, not delivered guarantees. Open alternatives remain open. This documentation-only refresh adds no compiler or M-track feature and reports no fresh compiler test run.

Current references, linked relative to the root PDF:

- TODO.md: sole live queue and execution cursor
- README.md: project orientation
- docs/compiler-status.md: detailed implementation snapshot
- docs/project-analysis.md: dated findings and experiment rationale

The current Current-State Audit is a September 9, 2026 dated assessment of the same 6ec4ba9 baseline, not a live status authority. Its August 25 version is retained in git history; the v0.1 manual remains a historical artifact.

Executive Summary

Sol is a statically typed, memory-safe, effect-aware, verification-capable programming language designed around the reality that software is continuously modified by teams containing both humans and generative models. Its central optimization target is not keystroke count. It is **preserving intent and correctness while changing code**.

The target language combines a Rust-like safe implementation substrate, Koka/F*-style effect tracking, Dafny-style contracts, algebraic data types, explicit capabilities, structured concurrency, state-machine protocols, schema-aware data evolution, and a stable semantic representation for programmatic editing. It intentionally avoids unrestricted textual metaprogramming, unchecked exceptions, implicit nullability, ambient authority, and multiple equivalent syntactic idioms.

Sol targets progressive assurance. The implemented base combines nominal types, ownership, exhaustive matching, explicit effects, typed errors, callable runtime contracts, and checked refined construction. A future proof layer adds logical normalization, ghost state, and solver-backed discharge of obligations, including loop invariants and termination. Normal application code need not become a theorem-proving exercise. Runtime checks are useful before proof, but do not prove all executions.

The compiler is intended to expose a canonical typed semantic graph, stable declaration identities, machine-readable diagnostics, constrained repair actions, proof caches, and behavior-change reports. These interfaces let tools patch declarations and obligations rather than brittle line numbers. Humans retain authority over intent, architecture, accepted effects, and proof policy.

IMPLEMENTED At 6ec4ba9, the C17 edition-2027 bootstrap has bounded reference execution over owning typed IR, explicit trusted hosting, `sol run` and authored Boolean `sol test`, runtime callable CHECK policy, direct checked refined construction, deterministic cleanup, package-local stable top-level IDs, structured diagnostics, and selected versioned inspection projections. E1-E6 and P1 are complete. Separate experimental owners provide frozen-E6 callable CFG MIR, monomorphic planning and concrete materialization, representation recipes, target layout, and source-independent semantic operations/predicate bodies through P2.6. These owners are not the CLI execution pipeline and impose finite callable-closure restrictions beyond interpreter support.

P2 remains open: symbols/linkage (P2.7) precede the first-user charter (M1), then the concrete-program freeze (P2.8). Production runtime ABI, backend emission, `sol build`, full public IR, SMT discharge, and semantic patch tooling remain absent. Loop/decreases templates are runtime-erased and unresolved; unreachable obligations are not proved. No M proposal is implemented by this refresh.

The proposed first focus is capability-restricted hosted business logic or validation; M1 still selects the user, workload, and host boundary. Existing projections and interpreter checks can support a small maintenance experiment without full IR or SMT. Safer or cheaper human/AI maintenance remains an unmeasured hypothesis, not a delivered benefit.

Key Decisions

Implemented decisions and longer-term proposals are distinguished below; the latter are not first-delivery requirements.

Area	Decision
Execution	Reference interpreter delivered; WebAssembly is the first planned production target over the frozen E6 core. No backend or Component Model integration yet. Native output and a possible VM/REPL are deferred long-term options.
Memory	Affine ownership with inferred borrowing and lexical/explicit regions; ordinary code requires no tracing garbage collector.
Types	Nominal algebraic types, distinct/refined types, bounded generics and traits are implemented. Broader numerics/units and dependent relationships remain target design.
Effects	The target is general row polymorphism; the bootstrap implements closed normalized finite rows, one callback-inferred free-function row parameter, and exact capability-sensitive effects.
Failure	Typed <code>Result</code> and declared <code>panic</code> are implemented; a future verified profile would forbid outward panic. No verified profile exists yet.
Verification	Typed templates, callable runtime CHECK, and direct refined construction are implemented. Loop/decreases/unreachable obligations remain unresolved; logical normalization, SMT, and proof discharge are future work. Pure does not mean total or proof-eligible.
Concurrency	Proposed structured tasks, cancellation, race-safe sharing, actors/channels, protocols, transactions, and workflows remain deferred.
Metaprogramming	Typed derives and semantic transforms remain proposed; unrestricted token macros are a non-goal.
AI interface	Package-local top-level semantic IDs, diagnostics, and selected inspection projections are implemented. Full public IR, semantic patches, change reports, and the bounded M experiment remain proposals.
Compatibility	Selected inspection schemas are versioned today. Schema evolution, API fingerprints/migrations, dependency-qualified identity, and FFI remain proposed.

Contents

Document Control	2
Executive Summary	3
Key Decisions	4
1 Vision, Scope, and Design Principles	8
1.1 Problem Statement	8
1.2 Product Vision	8
1.3 Primary Goals	8
1.4 Non-Goals	9
1.5 Design Principles	9
1.6 Target Users and Domains	9
2 Language Overview and Canonical Source Model	10
2.1 A Small Target Example	10
2.2 Lexical and Formatting Rules	10
2.3 Declaration Ordering	11
2.4 Expressions and Statements	11
2.5 Bindings and Mutation	11
2.6 Functions and Methods	12
2.7 Modules and Visibility	12
3 Type System and Data Modeling	13
3.1 Type-System Strategy	13
3.2 Primitive Types	13
3.3 Records, Enums, and Tuples	13
3.4 Distinct and Refined Types	14
3.5 Optionality and Absence	15
3.6 Generics, Traits, and Constraints	15
3.7 Units, Dimensions, and Currency	15
3.8 Collections and Views	16
3.9 Pattern Matching and Exhaustiveness	16
3.10 Type Inference Boundaries	17
4 Ownership, Regions, and Resource Safety	18
4.1 Ownership Model	18
4.2 Borrow Categories	18
4.3 Regions and Arenas	18
4.4 Destructors and Cleanup	19
4.5 Interior Mutability and Sharing	19
4.6 Unsafe Code	19
4.7 Real-Time and Allocation Profiles	19
5 Effects, Capabilities, and Failure Semantics	21
5.1 Why Effects Are in Function Types	21
5.2 Effect Rows and Polymorphism	21
5.3 Capabilities	22
5.3.1 Authority roots and may-origin sets	22
5.3.2 Authority-preserving returns and wrappers	22
5.4 Effect Handling	22
5.5 Typed Errors	23
5.6 Panic and Unreachable States	23
5.7 Cancellation, Timeout, and Retry	24
5.8 Standard Effect Categories	24
6 Contracts, Refinements, and Verification	25
6.1 Verification Philosophy	25
6.2 Preconditions and Postconditions	25
6.2.1 Bootstrap contract resolution and typing	25
6.3 Invariants and Loops	26

6.4	Ghost State and Erasure	26
6.5	Proof Modes	27
6.6	Runtime Contract Fallback	27
6.7	Specifications and Generated Tests	27
6.8	Solver Discipline	28
6.9	Cost and Resource Contracts	28
7	Concurrency, Protocols, Transactions, and Durability	30
7.1	Structured Concurrency	30
7.2	Race Safety	30
7.3	Actors and Channels	30
7.4	Protocol Types and State Machines	31
7.5	Transactions	31
7.6	Durable Workflows and Sagas	31
7.7	Atomicity and Concurrency Verification	31
8	Modules, Schemas, Interoperability, and Runtime Model	33
8.1	Packages and Editions	33
8.2	Stable Semantic Identities	33
8.3	Application Entrypoint	33
8.4	Trusted Interpreter Host Profile	34
8.5	Interpreter Application Command	35
8.6	Canonical Typed Interpreter IR	35
8.6.1	Compiler-Internal Reference Interpreter	37
8.7	Schema Definitions and Evolution	37
8.8	API Compatibility	38
8.9	Foreign Function Interface	38
8.10	WebAssembly Components	38
8.11	Runtime Profiles	38
8.12	ABI and Layout	39
9	AI-Native Development and Semantic Editing	40
9.1	Design Objective	40
9.2	Intent Blocks	40
9.3	Semantic Patch Language	40
9.4	Machine-Readable Diagnostics	41
9.5	Change-Oriented Compilation	41
9.6	Context Bundles	41
9.7	Trust and Approval Model	41
9.8	Canonical Intermediate Representation	41
10	Compiler Architecture, Toolchain, and Developer Experience	43
10.1	Compiler Pipeline	43
10.2	Front End	43
10.3	Verification Engine	43
10.4	Sol IR and MIR	43
10.5	Backends	44
10.6	Incremental Compilation	44
10.7	Command-Line Tool	44
10.8	Language Server and IDE	45
10.9	Build Reproducibility	45
10.10	Documentation	45
11	Standard Library, Ecosystem, and Security	46
11.1	Standard Library Principles	46
11.2	Library Layers	46
11.3	Text, Time, and Randomness	46
11.4	Networking and Services	46
11.5	Logging, Metrics, and Tracing	46
11.6	Package Registry and Supply Chain	46
11.7	Security Model	46
11.8	Reflection and Serialization Safety	47

12 Implementation Roadmap, Risks, and Open Questions	48
12.1 Recommended Implementation Strategy	48
12.2 Executable Bootstrap Baseline	48
12.2.1 Current Production Internals: P1 through P2.6	49
12.2.2 Predicate Bodies and Finite Closure	50
12.2.3 Remaining Boundary	50
12.3 Phased Roadmap	50
12.4 Minimum Viable Sol	51
12.5 Major Risks	52
12.6 Open Design Questions	52
12.7 Success Criteria	53
13 Worked Reference Examples	54
13.1 HTTP Handler with Explicit Authority	54
13.2 Idempotent Inventory Reservation	54
13.3 Embedded Controller	56
13.4 Protocol-Typed Database Session	56
13.5 Semantic Refactor	57
Appendix A. Syntax and Grammar Sketch	58
Appendix B. Standard Effect Taxonomy	60
Appendix C. Diagnostic and Patch Schemas	61
C.1 Diagnostic Envelope	61
C.2 Patch Envelope	61
Appendix D. Comparative Positioning	62
D.1 Closest Current Language	62
Appendix E. References and Design Influences	63
Closing Statement	64

READING PATH Readers evaluating the concept should begin with Sections 1, 2, 5, 6, and 9. Compiler implementers should continue through Sections 10 and 12. Section 12.2 and the status callouts distinguish the executable bootstrap from the target design. Worked examples are normative enough to expose proposed syntax inconsistencies, but only examples explicitly labeled bootstrap are expected to compile today.

1 Vision, Scope, and Design Principles

1.1 Problem Statement

Most mainstream languages were designed around a human author entering text and a compiler deciding whether it can execute. Modern development has a different bottleneck. Large programs are maintained through repeated modifications under incomplete context, often across repositories, schemas, services, and toolchains. Generative models amplify both the rate of change and the cost of ambiguity.

The most damaging errors rarely reflect an inability to express an algorithm. They fail to preserve hidden assumptions: whether a value can be absent, a function performs I/O, an operation is atomic, a resource escapes, a retry is idempotent, an old reader accepts a new schema, or a refactor alters observable behavior. Conventional languages often leave these facts in comments, naming, framework behavior, or tribal knowledge.

Sol treats these facts as language semantics. Inputs and outputs alone do not describe a function; declared effects, errors, ownership, contracts, cost constraints, and protocol transitions are also interface. The compiler maintains a semantic graph tools can query without reconstructing meaning from text.

1.2 Product Vision

The long-term vision spans ordinary services, systems tools, embedded control, libraries, and WebAssembly components without requiring formal-methods expertise. This is not the initial delivery scope. The proposed first domain is a capability-restricted hosted business-logic or validation component, selected by M1. The same codebase should eventually permit stronger contracts and proofs incrementally; testing the maintenance thesis does not require implementing that whole vision.

The intended loop is: state intent and constraints; implement inside a narrow semantic envelope; compile to receive structured obligations; apply a semantic change; inspect a behavior-change report. The safe, reviewable path should be easier than the clever path.

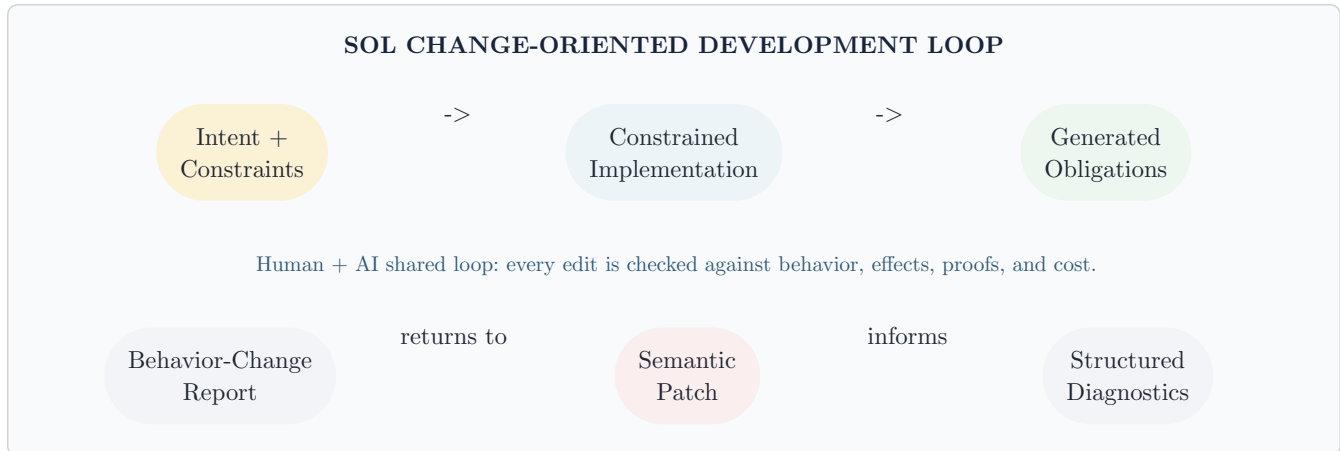


Figure 1: Sol optimizes the change loop, not merely the authoring step.

1.3 Primary Goals

- **Local semantic clarity.** A declaration reveals nullability, mutation, effects, failure, ownership, and contracts without distant configuration.
- **Invalid-state resistance.** Types represent domain states, protocols, and units rather than correlated booleans and comments.
- **Change safety.** Tooling reports behavior, effect, schema, cost, and proof changes caused by a patch.
- **AI-legible structure.** The compiler exposes canonical identities, structured diagnostics, and valid repair classes.
- **Progressive verification.** Ordinary code compiles quickly; high-assurance code states and proves stronger properties.
- **Predictable performance.** Zero-cost abstractions, deterministic destruction, bounded allocation profiles, and visible low-level escape hatches remain possible.
- **Interoperability.** Sol enters existing systems through C, WebAssembly components, generated bindings, and schema-defined services.
- **Canonicity.** Formatting and syntax minimize semantically meaningless variation.

1.4 Non-Goals

- Sol does not maximize terseness, code-golf expressiveness, or syntactic customization.
- Sol 1.0 will not expose unrestricted dependent types throughout ordinary programs.
- Sol does not guarantee every terminating program can be proved automatically; annotations, lemmas, or runtime checks may be required.
- Sol does not hide distributed-system failure behind transparent remote objects.
- During pre-1.0 work, source compatibility does not override semantic clarity.
- Safe code cannot acquire filesystem, network, process, clock, random, database, or unsafe-memory authority implicitly.

1.5 Design Principles

Principle	Consequence
Intent is part of the program	Contracts, effects, protocols, schema policy, and preservation constraints are versioned beside implementation.
Public interfaces are explicit	Inference reduces local annotation, but exported types and effects have one normalized form.
One obvious form	Canonical formatting minimizes noise; mandatory global declaration sorting is under reconsideration, not implemented policy.
Capability before convention	I/O requires explicit capability authority. Declared effects describe behavior but never grant permission; imports grant names only.
Proofs must compose	Verification is modular, cached, and bounded by interfaces; whole-program SMT is not normal.
Escape hatches are visible	unsafe , unchecked FFI, raw memory, nondeterminism, and assumptions are explicit and auditable.
Diagnostics are an API	Compiler output is structured, stable, and carries machine-actionable repair alternatives.
Source is not the sole truth	A canonical typed semantic graph is a supported artifact, not an implementation accident.

1.6 Target Users and Domains

Long-term candidate domains include backend services, command-line tools, systems components, embedded and real-time control, security-sensitive libraries, data pipelines, and WebAssembly components. They impose different ecosystem and assurance requirements, so they are not simultaneous first-user commitments. M1 must choose one hosted component and its protected maintenance tasks, rather than treating every missing language family as foundational.

The initial language is not optimized for browser UI frameworks, highly dynamic scripting, symbolic mathematics, or GPU kernels, though it should interoperate with them. A managed profile may improve desktop ergonomics later, but the baseline object model cannot depend on tracing collection.

2 Language Overview and Canonical Source Model

2.1 A Small Target Example

```

module inventory.reservation
  use commerce.OrderId
  use inventory.{ItemId, Quantity}
  use time.Instant

  enum ReservationError {
    item_not_found(item: ItemId)
    insufficient_stock(item: ItemId, requested: Quantity, available: Quantity)
  }

  record Reservation {
    order: OrderId
    item: ItemId
    quantity: Quantity where quantity > 0
    created_at: Instant
  }

  public transaction reserve(
    inventory: capability Inventory,
    clock: capability Clock,
    order: OrderId,
    item: ItemId,
    quantity: Quantity where quantity > 0,
  ) -> Result<Reservation, ReservationError>
  effects { database.transaction<inventory> clock.read<clock> }
  ensures {
    success => inventory[item].available
      == old(inventory[item].available) - quantity
    failure => inventory[item].available
      == old(inventory[item].available)
  } {
    let stock = inventory.lock(item)
      .or_error(ReservationError.item_not_found(item))?
    require stock.available >= quantity else {
      return ReservationError.insufficient_stock(
        item = item, requested = quantity, available = stock.available,
      )
    }
    stock.available -= quantity
    return Reservation { order, item, quantity, created_at = clock.now() }
  }

```

Listing 1: Representative target-design declaration with domain types, effects, a transaction boundary, and postconditions.

Modules and imports are explicit. Data types are algebraic and nominal. Nullability is `Option<T>`; recoverable failure is `Result<T, E>`. This creation-only target example receives inventory and clock authority explicitly. Its provider-owned state projections, locking, atomic rollback, inline refinements, and transaction syntax are assumptions, not bootstrap APIs. Explicit imports and direct nominal refined construction are implemented in narrower forms; this full listing is not accepted bootstrap input. Section 13’s retry example separately distinguishes replay from creation.

2.2 Lexical and Formatting Rules

- UTF-8 source is supported; public identifiers SHOULD use a restricted profile against confusables.
- Keywords and standard-library identifiers are lowercase ASCII. Types use UpperCamelCase; values, fields, modules, and effects use snake_case.
- Blocks use braces. Semicolons are absent. Canonical newlines separate statements.
- Indentation is four spaces; canonical formatting rejects tabs.
- Multiline arguments, fields, and variants require trailing commas; single-line forms omit them.
- One formatter produces stable output for each edition and defines canonical source representation.

- Comments use `//`, `/* ... */`, and `///`; nested block comments are supported.

CANONICAL SOURCE The compiler accepts a narrow grammar and `sol fmt` produces a normalized spelling. Canonicity reduces diff noise, simplifies generated patches, and stabilizes semantic hashes.

IMPLEMENTED The bootstrap formatter accepts syntactically valid edition-2027 files and deterministic directory packages. It preserves source token/comment bytes, source order, commas, and deliberate hard line breaks while normalizing spaces, four-space indentation, LF line endings outside comments, blank-line runs, final newline, braces, operators, delimiters, generic/effect angles, and comparisons. Output is re-lexed, reparsed, checked for token preservation, and formatted again for byte idempotence. `sol fmt` writes through a staged package transaction; `--check` is read-only and `--stdout` accepts one file. Malformed files are never rewritten. Width-based reflow, trailing-comma insertion/removal, import/declaration sorting, and comment/semantic reordering are future work.

2.3 Declaration Ordering

One proposed organization is module; edition/features; imports; public types; private types; constants; capabilities; functions; protocol/transaction declarations; specifications; tests. Mandatory global sorting is under reconsideration: locality and review churn matter, and stable identity need not require sorting. The implemented formatter preserves declaration and token order and does not authorize semantic reordering. The following is an illustrative organization, not a currently enforced ordering rule.

```
module billing.transfer
edition 2027
use identity.AccountId
use money.Money
public enum TransferError { ... }
public record TransferReceipt { ... }
const maximum_transfer = 100_000 USD
public function transfer(...) -> ...
effects { ... }
requires { ... }
ensures { ... } {
    ...
}
spec transfer { ... }
test "same-account transfers fail" { ... }
```

Listing 2: Illustrative target file organization, not mandatory sorting.

2.4 Expressions and Statements

Sol is expression-oriented but distinguishes effectful statements where reviewability improves. `if`, `match`, and blocks produce values. Assignment, resource acquisition, spawning, transaction control, and capability operations are statements. A block's final expression is its value.

```
let label = match account.state {
  active => "Active"
  frozen(reason) => "Frozen: {reason}"
  closed(at) => "Closed at {at}"
}
let fee = if amount >= 1_000 USD { 0 USD } else { 2 USD }
```

Listing 3: Exhaustive matching and expression-valued control flow.

2.5 Bindings and Mutation

- `let` creates an immutable binding.
- `var` creates a mutable local and is prohibited in pure-expression contexts.
- Records are immutable unless accessed through an exclusive mutable borrow or owned `modify` scope.
- Mutation syntax remains ordinary once exclusivity is proven.
- `var name: Type` is uninitialized and must be assigned on every continuing path before use.
- Arithmetic compound assignment is checked `Int64` read-modify-write in the bootstrap.

```

let original = account
modify account {
  account.balance += deposit
  account.updated_at = clock.now()
}
// `original` is an immutable value, not an alias to modified storage.

```

Listing 4: Explicit mutation scope.

2.6 Functions and Methods

Free functions are primitive. Method syntax is type-directed sugar when the first parameter is `self`. Extension methods are namespaced imports, not global monkey patches. Functions are nonvirtual except through trait values. Overload resolution is deliberately limited.

The implemented interpreter evaluates named call arguments in canonical formal-parameter order, not their written operand order. Reordering a signature can therefore reorder effects even if named calls look unchanged. Alternatives remain a future explicit decision; this refresh preserves the current rule. Bootstrap method dispatch is the bounded coherent trait subset described in Section 3, not the target extension-method model.

```

public function Money.add(
  self: Money<Currency>, other: Money<Currency>,
) -> Money<Currency>
effects { pure } {
  return Money(self.minor_units + other.minor_units)
}
let total = subtotal.add(tax)

```

Listing 5: Method syntax as a projection of an ordinary function.

2.7 Modules and Visibility

A module is a semantic namespace and compilation unit, not necessarily one file. Packages define a public module tree. Visibility is private, module, package, or public. Friend access is absent; cross-package internal access requires an explicit capability or separately versioned package.

Imports never globally change method lookup. Wildcards are rejected in library source and permitted only in REPL/test scopes. External names in public signatures carry canonical fully qualified identities in the semantic graph even when source uses imports.

IMPLEMENTED For directory `sol` check, the bootstrap recursively discovers regular `.sol` files in deterministic bytewise path order as one dependency-free package. Each file declares its module, multiple files may contribute to one module, and private declarations are visible throughout that module. File-scoped `use module.path.Symbol` imports resolve one public declaration and feed package-wide type, generic-bound, trait, effect, and contract checking. Import cycles are allowed because the subset has no top-level initialization. Direct file checking retains the earlier ignored-import compatibility behavior. Top-level semantic IDs survive source ordering and file placement changes; explicit public stable tokens support package-local rename/move identity. Manifests, external dependencies and dependency-qualified IDs, aliases, grouped/wildcard/relative imports, re-exports, separate private/module/package visibility, and imported extension-method search remain future work.

3 Type System and Data Modeling

3.1 Type-System Strategy

The target uses a layered static type system. A decidable base provides nominal algebraic types, generics, traits, affine ownership, effect rows, and exhaustiveness. Refinements add predicates over values and state. Proof terms, propositions, lemmas, and solver-guided obligations form a proof layer. Crossing to weaker layers requires erasure or a verified boundary.

USABILITY DECISION Routine syntax does not expose unrestricted full-spectrum dependent typing. Value-dependent constraints are admitted where obligations and tooling remain clear; advanced propositions stay in the proof sublanguage.

IMPLEMENTED The bootstrap implements first-order type parameters on records, enums, distinct/refined declarations, and free functions plus one trailing effect-row parameter on free functions. Parameters are rigid declaration-owned identities while templates are checked. Built-in and user applications are invariant and canonically interned by constructor identity plus ordered exact arguments; `Option<T>` and `Result<T, E>` retain their exact one- and two-argument identities. Type positions require complete explicit application. Function calls use either all explicit type arguments or argument-only recursive structural inference; callback arguments infer the bounded effect parameter. Free-function type parameters may carry one inline trait bound, checked after inference and forwarded symbolically through bounded generic calls across modules. Reference execution carries substitutions and evidence; P2.2/P2.3 separately plan and materialize canonical monomorphic instances within the frozen finite closure. Defaults, variance, higher-kinded/lifetime/const parameters, multiple bounds, general effect constraints, generic capabilities/members, polymorphic recursion, and separately compiled dependency instantiation remain future work.

3.2 Primitive Types

Category	Types and behavior
Integers	<code>Int8</code> through <code>Int128</code> , unsigned equivalents, pointer-sized <code>IntSize/UIntSize</code> , and arbitrary <code>BigInt</code> ; overflow policy is explicit.
Floating point	<code>Float16</code> , <code>Float32</code> , and <code>Float64</code> with IEEE-oriented semantics and explicit NaN behavior.
Decimal	Fixed-scale <code>Decimal<Scale></code> and arbitrary decimal; no implicit binary-float conversion.
Boolean	<code>Bool</code> ; domain booleans are discouraged when enums communicate meaning.
Text	<code>Text</code> is immutable Unicode scalar text; <code>Bytes</code> is immutable bytes; integer text indexing is unsupported.
Characters	<code>Rune</code> is a Unicode scalar; grapheme operations require explicit locale/data dependencies.
Never/unit	<code>Never</code> has no values. <code>Unit</code> has one and prints <code>()</code> .

Only `Int64`, `Bool`, `Text`, `Unit`, and `Never` from this primitive catalog currently participate in bootstrap expression typing.

3.3 Records, Enums, and Tuples

Records are nominal products. Enums are nominal tagged unions with typed payloads. Matching a closed enum is exhaustive. Public enums are closed by default; an explicitly open enum admits unknown future variants and requires a fallback. The bootstrap substitutes rigid type parameters through generic record construction/access and generic enum construction/pattern bindings.

```

record Address { street: Text, city: Text, region: Text, postal_code: PostalCode }
enum PaymentResult {
  approved(receipt: Receipt)
  declined(reason: DeclineReason)
  retryable(error: GatewayError, after: Duration)
}
open enum WireMessage { request(Request) response(Response) }

```

Listing 6: Nominal products, closed sums, and opt-in extensible wire enums.

Tuples are structural products with 2 through 16 ordered elements. `(a, b)` and `(a, b,)` construct the same tuple type; `()` remains `Unit`, `(value)` remains grouping, and singleton `(value,)` syntax is reserved. A tuple field is selected only by its canonical zero-based decimal ordinal `.0` through `.15`; leading-zero spellings such as `.00`, named fields, and out-of-range ordinals are rejected. Tuple types use the corresponding `(A, B)` syntax, participate structurally in signatures, generic inference, substitution, equality eligibility, ownership, and representation, and cannot be trait implementation targets. Evaluation and cleanup follow element source order. Numeric projections are ordinary readable, movable, borrowable, and assignable places, including nested and computed-root chains. Public returns with independently meaningful fields SHOULD use named records.

```

function split(value: Int64) -> (Int64, Bool) {
  return (value, value == 0,)
}

function first(pair: (Text, Int64)) -> Text {
  return pair.0
}

```

Listing 7: Structural local products and projections.

IMPLEMENTED The bootstrap parses, formats, relocates, type-checks, lowers, ownership-checks, and interprets structural tuples of arity 2 through 16, including exact-arity recursive tuple patterns. Tuple applications are canonically interned without a nominal definition. Equality is recursive and excludes any tuple containing a capability, function, bound operation, unresolved parameter, or other non-equality element. Copy/affine classification and partial moves recurse by element; exact projection assignment restores moved paths. Capability and operation provenance crosses tuple construction, aliases, and projection, while authority-bearing projected replacement remains conservatively restricted.

3.4 Distinct and Refined Types

A distinct type shares representation but not implicit interchangeability. A refined type carries a predicate. Construction generates a proof obligation or runtime validation according to profile.

```

type UserId = distinct Uuid
type AccountId = distinct Uuid
type Age = refined Int where 0 <= self <= 150
type EmailAddress = refined Text using email.is_valid
type NonEmpty<T> = refined List<T> where self.length > 0

```

Listing 8: Domain identity and value refinement.

Widely instantiated predicates must be solver-friendly and effect-free. Refinements do not alter representation unless a validation tag or witness is retained.

IMPLEMENTED The bounded bootstrap supports generic and nongeneric type `Name = distinct Representation` declarations as fresh nominal identities with explicit one-argument construction and no implicit representation conversion. `type Name = refined Representation where predicate` binds contextual `self` at the representation type, requires `Bool`, applies finalized contract-purity rules, and lowers exactly one deterministic type-owned obligation. Generic declarations are checked once under rigid parameters and generic declared-type construction requires explicit arguments. Public declarations import across package modules, and exact closed declared types may be trait implementation targets. Capability-containing or cyclic representations are rejected. Every direct refined construction evaluates its predicate under ordinary interpreter limits before creating the nominal wrapper, independent of callable contract policy; false reports a structured refinement violation and runtime predicate failures retain their code. Predicate assumptions, static proof

discharge, using predicates, inline refinements, projection/destructuring, and refinement-aware exhaustiveness remain future work.

3.5 Optionality and Absence

There is no implicit null. `Option<T>` has `some(T)` and `none`. Nullable foreign pointers must be wrapped, checked, or represented by a foreign handle.

```
let nickname: Option<Text> = user.nickname
match nickname {
  some(value) => greet(value)
  none => greet(user.display_name)
}
```

Listing 9: Explicit absence.

3.6 Generics, Traits, and Constraints

Generics are reified in semantic metadata; the frozen production path plans canonical monomorphic instances before the first Wasm backend. Dictionaries or shared instantiations for other ABI/code-size policies remain longer-term options. Conformance is explicit and coherent; the broader package design proposes that a package implement a trait for a type only if it owns the trait or type.

```
trait Display {
  function display(self: Self) -> Text
  effects { pure }
}
implementation Display for Int64 {
  function display(self: Self) -> Text effects { pure } {
    return "number"
  }
}
public function render<T: Display>(value: T) -> Text
effects { pure } {
  return value.display()
}
```

Listing 10: Trait-bounded generic function.

Traits may declare associated types, constants, and effects in the target language. Trait objects require explicit `dynamic Trait` and define an ABI-stable dispatch boundary. Arbitrary overlapping specialization is absent.

IMPLEMENTED The bounded bootstrap supports nongeneric traits, exact closed implementations, one inline bound per free-function type parameter, and immediate type-directed dot calls. Requirements and implementation methods use `self: Self`, explicit canonical parameter/result types, and exactly matching closed authority-independent effect rows. One coherent implementation provides every required method exactly once. Concrete calls select an exact implementation; symbolic calls on rigid parameters select their declared requirement, and checked metadata carries that choice into effect and contract purity validation. Generic traits and methods, defaults, method contracts or authority clauses, generic/blanket/conditional implementations, associated items, inheritance, multiple bounds, method values, trait objects, specialization, imported extension lookup, and dependent method effects remain future work.

3.7 Units, Dimensions, and Currency

Physical dimensions and currencies participate in numeric types. Dimensional algebra rejects incompatible addition or comparison. Conversions are explicit and can be zero-cost for known ratios.

```

let distance: Meter = 100 meters
let elapsed: Second = 9.58 seconds
let speed: Meter / Second = distance / elapsed
function kinetic_energy(mass: Kilogram, velocity: Meter / Second) -> Joule {
    return 0.5 * mass * velocity^2
}
let invoice_total: Money<USD> = 125.00 USD
let euro_total = exchange.convert(invoice_total, to = EUR)?

```

Listing 11: Dimensional analysis and currency identity.

Dimension metadata erases when representation is unambiguous. Currency conversion is effectful because rates depend on source and timestamp. Units are not implemented.

3.8 Collections and Views

Type	Semantics
Array<T, N>	Fixed contiguous owned array; N may be a compile-time natural.
Vector<T>	Growable contiguous owned collection.
List<T>	Persistent immutable list optimized for structural sharing.
Map<K, V>	Concrete ordered or hashed maps expose ordering guarantees.
View<T>	Read-only borrowed sequence with lifetime inferred from its source.
Slice<T>	Contiguous borrowed range; mutable slices require exclusivity.
Stream<T, E>	Effectful asynchronous sequence with visible effect and error types.

Operations distinguish eager allocation, lazy iteration, and effectful streaming. A transformation cannot silently become allocating without affecting a cost contract or change report.

3.9 Pattern Matching and Exhaustiveness

Patterns recursively combine `_`, `true/false`, bindings, positional enum variants, structural tuples of arity 2 through 16, and nominal records written `Type { field, field = pattern }`. Record fields may appear in any order and may be omitted; omission is structural wildcarding, not a rest pattern. A top-level bare payloadless variant retains the existing `variant` spelling. Within another pattern a bare identifier is always a binding, so a nested payloadless enum variant is written `variant()`. If a nested binding's spelling collides with a variant of its expected enum, the compiler diagnoses the collision rather than silently creating an accidental catch-all.

```

match message {
    delivered(Receipt { status, route = (primary(), hops) }) if hops > 0 => status
    delivered(Receipt { status }) => status
    pending => "pending"
}

```

Listing 12: Recursive destructuring with a pure guard.

Guards use `pattern if expression => value`, have exact type `Bool`, and may refer to bindings introduced by the pattern. They pass the existing strict contract-purity firewall: effects, mutation, control transfer, propagation, handlers, panic, and divergence are forbidden. Because a false guard falls through, neither a pattern binding nor an outer affine local may be consumed by a guard.

The scrutinee is evaluated and consumed exactly once. Each candidate first performs its structural test and creates bindings, then evaluates its guard if present. A false guard cleans those candidate bindings and continues; a true guard evaluates the body. Bindings are cleaned in deterministic reverse order on false-guard falthrough and every arm exit.

Usefulness and exhaustiveness recurse through `Bool`, closed and open enums, records, tuples, and generic substitution. Guarded arms add no coverage; only prior unguarded coverage can make a later arm structurally unreachable. An open enum therefore requires an unguarded fallback. Finite-inhabitation analysis recognizes recursive uninhabited closed enums, so an empty finite/uninhabited closed enum permits a zero-arm match whose type is `Never`. Analysis is deterministic and bounded: pattern depth is 64, usefulness takes at most 8,192 steps, and finite-inhabitation analysis takes at most 1,024 steps at depth 64.

IMPLEMENTED The bootstrap implements exactly the recursive pattern and guard subset above in owning typed IR, with independent IR validation and reference-interpreter execution. It does not implement rest, or, range, refined, protocol-state, or reference patterns. Refined patterns remain roadmap item 50 and protocol-state patterns remain item 59.

3.10 Type Inference Boundaries

The target infers locals, generic arguments, borrow regions, and effect rows when unambiguous. Public declarations, trait members, recursive functions, FFI, serialized fields, and persistent schemas require canonical types. The bootstrap requires declared parameter and return types; its private function effect inference, including recursive functions, is described in Section 5.

4 Ownership, Regions, and Resource Safety

4.1 Ownership Model

Every nontrivial resource has an owner. Values are affine by default: consumed at most once unless `Copy` or borrowed. Moving invalidates the source. Destruction is deterministic at scope exit unless ownership returns, transfers, or enters a longer-lived region.

The surface aims to expose less lifetime syntax than Rust while retaining comparable safety. Most borrow regions derive from lexical structure, effect boundaries, and escape analysis. Explicit regions appear where an API relates input and output lifetimes.

```
function first<'data, T>(values: View<'data, T>) -> Option<View<'data, T>>
effects { pure } {
  ...
}
```

Listing 13: Explicit region only where output borrows from input.

IMPLEMENTED The bootstrap checks definite initialization, path-sensitive affine moves, mutable replacement, and callable-scoped shared/exclusive loans after typed IR lowering. `var name: Type` introduces an unbound authority-free slot; a whole-local assignment must initialize it on every reachable continuing path before reads, borrows, projection, compound assignment, or `modify`. Branch/match joins intersect initialization and exclude terminating paths. Operational projections are local-rooted record fields and tuple ordinals. Paths overlap when they share a root and either projection path prefixes the other; sibling fields or tuple elements are disjoint. Moving an affine projection makes that path and its ancestors unavailable while preserving siblings. Exact projection or ancestor assignment can restore availability. Plain assignment analyzes the RHS first, rejects overlapping loans, records UPDATE use metadata, cleans a live old leaf exactly once, and fills a moved hole without old-value cleanup. `+=`, `-=`, `*=`, `/=`, and `%=` are checked `Int64` operations that read the target once and commit only after RHS evaluation, arithmetic, and path-step preflight succeed. `modify local { body }` grants lexical assignment and exclusive-call authority to one initialized immutable owned binding or owned parameter; it accepts only a whole local and a `Unit/Never` body and does not alter global mutability metadata. Projected mutation and whole replacement lacking exact top-level provenance conservatively reject authority-bearing/function/`Self`/unresolved-generic values because field-sensitive authority provenance is deferred. Source-defined calls implement successful copy-in/copy-out `inout`; bodyless host operations and top-level entries reject it. Statement-only loops use bounded ownership/initialization fixed points and exact per-edge cleanup. Index/dereference projections, field-by-field first initialization, authority-bearing uninitialized slots, escaping references, loop labels/values and obligation discharge, allocator APIs, lifetime generics, and user resource cleanup remain deferred. Current inspection is outer v3 with syntax/types v3; owning IR remains excluded.

4.2 Borrow Categories

Form	Meaning
<code>borrow T</code> parameter	Bootstrap shared callable access to a local-rooted local, record-field, or tuple-projection place; compatible shared loans coexist.
<code>inout T</code> parameter	Call-scoped exclusive access to a writable local-rooted record-field or tuple-projection place; supported source callables copy in and write back on success.
<code>Owned T</code>	Callee receives responsibility for destruction or transfer.
<code>shared T</code>	Explicit reference-counted/runtime ownership; traits govern task safety.
<code>handle<Resource></code>	Opaque capability-bearing resource with protocol operations and deterministic close.

4.3 Regions and Arenas

Lexical regions permit bulk allocation/destruction. Region values cannot escape unless independent or moved/copied into a longer-lived owner. Region allocation is a standard capability and may be forbidden in heap-free profiles.

```

region frame {
  let vertices = Vector<Vertex>.with_capacity(mesh.vertex_count, in = frame)
  build_vertices(mesh, into = vertices)
  renderer.draw(vertices.view())
}
// All frame allocations are reclaimed here.

```

Listing 14: Region-scoped allocation.

The preceding allocator-bearing region is target design. The bootstrap region label cannot be referenced and no allocator API exists. Its current role is a structured lexical boundary for ownership diagnostics and exact cleanup metadata. Normal calls may consume region locals because the bootstrap has no global storage or closures.

4.4 Destructors and Cleanup

Drop provides deterministic cleanup. Cleanup cannot report a recoverable error because another failure may be active. Fallible shutdown uses `close() -> Result<(), CloseError>`; **Drop** performs only best-effort nonblocking safety cleanup. Destructors have tightly constrained effects: network, arbitrary blocking, spawning, and visible logging require `unsafe/profile` permission.

This **Drop/close** model is target design. Bootstrap cleanup is only internal interpreter storage release: it is effect-free, invokes no Sol code or host operation, cannot replace an active diagnostic, and is not a destructor, finalizer, close operation, allocator action, or backend guarantee. An explicitly unstable interpreter request may synchronously observe local IDs and monotonic cleanup ordinals for tests; no pointer is exposed and no event occurs before canonical IR validation.

4.5 Interior Mutability and Sharing

Interior mutability uses explicit synchronization/cell types with visible effects. Safe cross-task sharing requires **Share**; transfer requires **Send**. Compiler-governed primitive ownership traits may be manually implemented only with `proof` or `unsafe`.

```

function lookup(cache: capability Cache, key: Key) -> Option<Value>
effects { sync.lock<cache> } {
  using read = cache.read()
  return read.get(key).copied()
}

```

Listing 15: Target synchronization provider passed as explicit authority.

4.6 Unsafe Code

unsafe is an effect and lexical block. It permits raw pointers, unchecked refined values, unchecked FFI, or asserted ownership facts. Every block declares assumed invariant and established safe property.

```

unsafe because {
  assume buffer points to `length` initialized bytes
  establish returned slice is valid for region `packet`
} {
  return Slice.from_raw_parts(buffer, length, in = packet)
}

```

Listing 16: Unsafe proof boundary.

Unsafe blocks enter the semantic graph and audits; changed assumptions invalidate dependent proof caches.

4.7 Real-Time and Allocation Profiles

Packages/functions may declare `general`, `bounded`, `heap_free`, or `real_time`. Profiles constrain allocation, blocking, synchronization, panic, and destructor behavior and compose through effects and cost contracts.

```
function control_tick(state: inout ControllerState, input: SensorFrame) -> ActuatorFrame
effects { pure }
resources {
  profile = real_time
  stack <= 2 KiB
  heap == 0 B
  worst_case_time <= 200 us
} {
  ...
}
```

Listing 17: Resource-constrained function.

5 Effects, Capabilities, and Failure Semantics

5.1 Why Effects Are in Function Types

Inputs and outputs do not reveal clock, randomness, database, network, locking, suspension, allocation, logging, panic, or unsafe behavior. These affect determinism, security, testing, retries, caching, and cost. Every function computation type therefore has an effect row.

Pure functions are the simplest case. An effectful function declares a normalized set. Local inference is permitted, but exported signatures and trait requirements print effects explicitly. A caller must possess compatible capability authority or handle the effect.

```
function calculate_total(cart: Cart, rules: PricingRules) -> Money<USD>
effects { pure }
function submit_order(
  order: PendingOrder,
  orders: capability Orders,
  gateway: capability PaymentGateway,
  clock: capability Clock,
  entropy: capability SecureRandom,
) -> Result<OrderId, SubmitError>
effects {
  database.transaction<orders>
  network.call<gateway>
  clock.read<clock>
  random.secure<entropy>
}
```

Listing 18: Pure and effectful target function types.

5.2 Effect Rows and Polymorphism

The target design has open, normalized, row-polymorphic effects. Ordering is canonical and does not imply execution order.

```
function map<T, U, effects E>(
  values: View<T>,
  transform: function(T) -> U effects E,
  allocator: capability Allocator,
) -> Vector<U>
effects { E memory.allocate<allocator> }
```

Listing 19: Target effect-polymorphic higher-order function.

IMPLEMENTED The bootstrap implements closed normalized rows plus a bounded, rank-1 row-polymorphic subset. A free function may declare one trailing **effects E** parameter, use it in structural callback parameter types, and include it in its explicit row. Calls infer the least closed authority-free argument from callbacks, currently containing only **panic** and **diverge**, union constraints across occurrences, and publish a deterministic instantiated call row. **pure** and **{}** are empty; private omitted rows still use SCC least-fixed-point inference. Explicit or multiple row arguments, all result-position row parameters, authority-dependent row capture, recursive row polymorphism, generic function callback coercion, and handler transformation of unresolved rows remain unsupported.

Recursive private inference computes a least fixed point over call-graph strongly connected components. Formal capability arguments and **Self** effects substitute at calls, including aliases and mutual/self recursion. Explicit rows are modular boundaries but are checked against every performed effect.

The bootstrap normal form distinguishes authority-free atoms, capability-parameter atoms, and member-relative **Self** atoms. Effect rows disclose behavior but do not grant permission. Exact unparameterized **panic** and **diverge** are the only compiler-defined authority-free atoms. Every other non-pure executable atom requires a lexical capability parameter, or **Self** on a capability member. Unparameterized resource effects and static-path authorities are rejected because the bootstrap has no package-global or runtime static authority root; importing a name never adds authority. Closed structural callback rows and inferred row arguments may contain only authority-free atoms because callback types cannot capture lexical capability

provenance. E3 hosting implements exact root/member grants and preflight for its bounded profiles. Broader host wiring, allocator APIs, unsafe/FFI gates, package policy, and production sandbox enforcement remain future work; the target listings above assume those providers.

5.3 Capabilities

An effect states what may happen; a capability value states which authority enables it. Capabilities can be passed, restricted, wrapped, and replaced in tests. Application wiring supplies authority; importing a module cannot create it.

```
capability PaymentGateway {
  function authorize(request: AuthorizationRequest)
    -> Result<Authorization, GatewayError>
  effects { network.call<Self> }
}
function checkout(gateway: capability PaymentGateway, order: Order)
  -> Result<Receipt, CheckoutError>
effects { network.call<gateway> } {
  ...
}
```

Listing 20: Capability-bearing service boundary.

5.3.1 Authority roots and may-origin sets

IMPLEMENTED Each capability value and bound operation carries an interned, sorted, duplicate-free, finite nonempty set of possible lexical capability-parameter roots. A runtime value still has one root; this set is conservative may-origin provenance. Immutable aliases, blocks, exact authority-preserving calls, restrictions, and wrappers preserve it. Reachable `if` and `match` branches union their sets; `Never` branches contribute nothing. Type and effect checking validate canonical root sets defensively.

For a parameter- or `Self`-dependent operation, the bootstrap conservatively expands one ordinary parameterized effect atom for every possible root. Explicit rows must contain every expansion; inferred rows union them, including recursively. This is finite closed-row expansion, not a row variable. Mixed provenance is accepted for ordinary calls and provider expressions, but declarations promising exact return authority and handler targets require singleton roots.

5.3.2 Authority-preserving returns and wrappers

An exact capability-returning function may state `authority { result derives_from parameter }`; a capability member may state `authority { result derives_from Self }`. The checker requires the actual return provenance to equal that singleton source, preventing a declared authority lie.

```
capability ReadFileSystem derives_from source: capability FileSystem {
  function read(path: Text) -> Text
  effects { filesystem.read<Self> } {
    return source.read(path)
  }
}
let restricted = ReadFileSystem { source = filesystem }
```

Listing 21: Nominal single-source capability wrapper.

Implemented wrappers are closed, nominal, and have exactly one private capability source. Every operation body is checked; construction requires exactly that source; derivation cycles are rejected; the wrapper does not subtype or coerce to its source. `Self` on both sides normalizes to the original root, allowing type-changing restriction without inventing authority.

5.4 Effect Handling

The target design permits selected algebraic handlers to interpret operations and remove or transform an outward effect. They suit deterministic clocks, test randomness, tracing, state, and domain effects, but cannot conceal distribution or violate ownership.

```

capability Clock {
  function now() -> Int64 effects { clock.read<Self> }
}
capability FixedClock {
  function now() -> Int64 effects { pure }
}
function deterministic(
  clock: capability Clock,
  fixed: capability FixedClock,
) -> Int64 effects { pure } {
  return handle clock.read<clock> with fixed {
    clock.now()
  }
}

```

Listing 22: Implemented exact capability-backed clock handler.

IMPLEMENTED Exact syntax is `handle effect.name<authority> with provider { body }`. The target is one parameterized atom, resolves to exactly one module-local capability member, and that source member's complete row is exactly the matching `{ effect.name<Self> }`. The target authority must have one known root because dynamic runtime authority matching is unsupported. The provider exposes the same operation name and exact parameter/result signature with a pure row.

Provider evaluation occurs outside the handler and its effects remain outward. The provider may have multiple possible roots because provider provenance is neither matched nor subtracted. Calls are handled deeply and lexically only when the instantiated operation name and authority root exactly match. Matching atoms are subtracted from the body's outward row, while other roots, residual effects, and call-graph edges remain. The type table retains source member, provider member, and singleton target root as runtime interception metadata. The handler preserves the body's type and value.

The compiler-internal reference interpreter implements this exact deep lexical interception over runtime capability-root identity. Production runtime/backend behavior remains future work.

FUTURE WORK General algebraic handlers, resumptions, effect-row transformation, static/unparameterized targets, multiple operations, and dynamic authority matching remain target design. The v0.1 `handle clock.read` with `FixedClock(...)` example was not valid bootstrap syntax and is superseded above.

5.5 Typed Errors

Recoverable failure uses `Result<T, E>`. Error types are algebraic and public. `?` is type-checked early return, not hidden unwinding. Conversion requires a declared function or variant relationship.

```

function load_config(filesystem: capability FileSystem, path: Path)
  -> Result<Config, ConfigError>
effects { filesystem.read<filesystem> } {
  let bytes = filesystem.read(path).map_error(ConfigError.read_failed)?
  let syntax = config.parse(bytes).map_error(ConfigError.invalid_syntax)?
  return config.validate(syntax)?
}

```

Listing 23: Typed error propagation.

The bootstrap types and executes exact `Result` applications and `ok/err` propagation with cleanup. Separate P2 owners plan/materialize bounded monomorphic programs and representation-aware propagation. This target filesystem/configuration listing additionally needs unimplemented hosting and library APIs; arbitrary interpreter programs are not all accepted by the finite production closure.

5.6 Panic and Unreachable States

`panic` represents a defect or violated invariant, not normal control flow. A function that may panic carries the effect unless unreachability is proved. Verified/safety profiles prohibit outward panic. Production policy may abort, terminate a task, restart a supervised actor, or trap.

IMPLEMENTED The bootstrap provides statement-only `panic message, unreachable because { proof },` and `require condition else { fallback }.` A panic message has exact type `Text`, is evaluated once, contributes the authority-free unparameterized `panic` effect, and terminates as `Never`. The reference interpreter returns a structured panic diagnostic rather than aborting and performs ordinary reverse lexical cleanup. An unreachable proof has exact type `Bool`, resolves in the current pre-statement lexical scope, passes the contract purity/forbidden-form firewall, and is erased from runtime effects, ownership transfer, cleanup, host calls, and step accounting. It emits one deterministic unresolved callable-owned obligation; no solver or discharge is implied. Reaching the statement at runtime is a distinct defensive error. A `require` condition has exact type `Bool`; its mandatory fallback block has exact type `Never`. The condition executes once. A true result continues as `Unit` without adding a refinement fact; false executes the fallback and propagates its terminating flow. Fallback-only moves do not pollute the continuing ownership state. Backend panic policy and obligation discharge remain future work.

```
unreachable because {
  state.validate() == false
}
```

Listing 24: Unreachability requires an obligation.

5.7 Cancellation, Timeout, and Retry

Cancellation and timeout are typed outcomes, not invisible exceptions. Suspending operations are cancellation-aware unless noncancellable. Retry requires an `Idempotent` proof or explicit compensation.

```
let result = within 500 ms { payment_gateway.authorize(request) }
match result {
  completed(value) => value?
  timed_out => return CheckoutError.gateway_timeout
  cancelled => return CheckoutError.cancelled
}
```

Listing 25: Explicit timeout and cancellation outcomes.

5.8 Standard Effect Categories

Category	Examples
Observation	<code>clock.read</code> , <code>environment.read</code> , <code>configuration.read</code>
Nondeterminism	<code>random.pseudo</code> , <code>random.secure</code> , <code>scheduler.nondeterministic</code>
I/O	<code>filesystem.read/write</code> , <code>network.call</code> , <code>console.write</code> , <code>process.spawn</code>
State	<code>state.local</code> , <code>database.read/write/transaction</code>
Concurrency	<code>task.spawn/suspend</code> , <code>sync.lock</code> , <code>channel.send</code> , <code>actor.message</code>
Resources	<code>memory.allocate</code> , <code>memory.pin</code> , <code>blocking</code> , <code>gpu.dispatch</code>
Failure	<code>panic</code> , <code>cancel</code> , <code>timeout</code>
Boundary	<code>unsafe</code> , <code>ffi.call</code> , <code>host.intrinsic</code>

6 Contracts, Refinements, and Verification

6.1 Verification Philosophy

Sol specifications should be executable enough to test and formal enough to prove. The target compiler turns refinements, contracts, invariants, coverage, ownership facts, protocol transitions, and resource bounds into obligations. Typing, linear analysis, abstract interpretation, symbolic execution, SMT, bounded protocol model checking, or proof terms discharge different classes.

Verification must remain modular. Packages verify against dependency interfaces. Proof caches key semantic declarations, assumptions, solver configuration, and imported contracts. Formatting cannot invalidate proof; changed effects or preconditions can.

IMPLEMENTED The bootstrap parses and resolves structured **requires** and **ensures** conditions, lowers deterministic semantic obligations after effect inference, and executes them in the reference interpreter under the **CHECK** policy. It does not normalize solver IR, invoke SMT, or discharge proofs.

6.2 Preconditions and Postconditions

```
function withdraw(account: Account, amount: Money<USD>) -> Account
effects { pure }
requires {
  amount > 0 USD
  account.state is active
  account.balance >= amount
}
ensures {
  result.id == account.id
  result.balance == account.balance - amount
  result.state == account.state
} {
  return account with { balance = account.balance - amount }
}
```

Listing 26: Function contract with **requires**, **ensures**, **result**, and immutable update.

A caller establishes preconditions; an implementation establishes postconditions. **old(expression)** denotes entry-state values. Postconditions can select success/failure outcomes. Ownership/effects infer frames, strengthened by **preserves** clauses in the target.

6.2.1 Bootstrap contract resolution and typing

Conditions are newline- or comma-separated and clauses occur in canonical order: **effects**, **requires**, **ensures**. Each condition resolves independently in a fresh declaration-signature scope. Parameters and declarations are visible; body locals, another condition's locals, and a derived wrapper's private source are not. Ordinary expression typing applies and the predicate **MUST** be **Bool**.

Contract calls are allowed only after their exact target effects are finalized and pure. This includes pure functions, pure capability operations, and closed pure callbacks. Effectful calls, **return**, **?**, and handlers are rejected in predicates.

For this bootstrap rule, **pure** means an empty finalized observable-effect row, not totality, absence of runtime failure, or automatic proof eligibility. The predicate firewall additionally rejects mutation and forbidden control forms. Even accepted pure predicates may fail checked arithmetic or exhaust deterministic limits; proof eligibility would also require a supported logical fragment, termination reasoning, and explicit trusted assumptions. Contract and guard propagation remain rejected with **SOL-CONTRACT-002**; P2.6 adds no new source behavior.

In ordinary postconditions, **result** has the declared return type. Outcome prefixes are available only on declarations returning **Result<T, E>**: a **success =>** condition binds **result: T**, while **failure =>** has no result binding. A prefixed condition on any other return type is rejected with **SOL-CONTRACT-005**. Each syntactic **old(expression)** creates its own typed entry-state snapshot, even if operands repeat. **old** is limited to postconditions; nested **old** and **old(result)** are rejected.

The public deterministic template table records sequential obligation identity, owner kind/identity, clause kind, outcome, predicate and **Bool** type, result availability/type, and contiguous snapshot identity/operand/type metadata. It is deliberately a template, not proof success.

IMPLEMENTED With interpreter policy **CHECK**, callable preconditions execute in source order before the body, every postcondition **old** operand is captured at entry, and applicable postconditions execute after a type-valid return and before copy-out. Unprefixed **result** denotes the complete return value; **success** selects only a successful **Result** and binds its payload; **failure** selects only an error **Result** and has no result binding. Bodyless hosted capability operations use the same contract framing. Predicates share ordinary step, value, text, call-depth, and host-call limits. A false predicate produces a dedicated require/ensure diagnostic at its source span, while panic and other runtime failures preserve their original code. Contract failure performs normal reverse cleanup exactly once; logical contract copies are not observable storage cleanup actions. **IGNORE** remains an explicit library policy. **sol run** and **sol test** select **CHECK**.

These sequential identities are deterministic for identical input but are not stable semantic identities across insertion, movement, or refactoring.

6.3 Invariants and Loops

Loops require sufficient invariants when correctness cannot be inferred. Total functions prove termination with **decreases**. Partial functions carry **diverge** and cannot serve as proofs or compile-time evaluation.

IMPLEMENTED The bootstrap accepts statement-only **loop { body }** and **while condition { body }**, plus payloadless final-in-block **break** and **continue** targeting the nearest loop. Either loop may carry at most one **invariant { ... }** clause followed by at most one **decreases { ... }** clause. An invariant clause is a nonempty newline- or comma-separated list of exact **Bool** expressions; **decreases** contains exactly one **Int64** expression. Specifications resolve in pre-body lexical scope and pass the contract purity firewall. They reject **result**, **old**, mutation, propagation, handlers, nested loops, and control transfer. A reachable backedge requires **decreases** unless the finalized owner row contains concrete unparameterized **diverge**; a symbolic row tail is not sufficient. Each invariant emits entry and preservation templates, and each measure emits nonnegative and strict-decrease templates, in deterministic lexical order. Templates retain exact callable/loop ownership, expression, type, and span in owning IR but are erased from execution, effects, ownership transfer, cleanup, and step accounting. The bootstrap does not discharge these obligations. Conditions are **Bool**; bodies are **Unit** or locally terminating. **while** always retains a condition-false exit. Bare **loop** falls through only through reachable breaks. Ownership computes a finite fixed point over initial entry plus fallthrough/continue backedges, joining unavailable paths by union and definite initialization by intersection, then joins reachable break states for the post-loop path. Runtime blocks unwind lexical locals on every edge and unregister per-iteration bindings before the next visit. Labels, exit payloads, loop values, and proof discharge remain deferred.

```
var index = 0
var sum = 0
while index < values.length
invariant {
  0 <= index <= values.length
  sum == values[0..index].sum()
}
decreases { values.length - index } {
  sum += values[index]
  index += 1
}
```

Listing 27: Loop invariant and termination measure.

6.4 Ghost State and Erasure

Ghost declarations exist only for specification/proof and cannot influence runtime branching, I/O, layout, or ABI unless materialized through proven reflection. The compiler erases them and checks noninterference.

```
ghost record TransferModel {
  total_before: Money<USD>
  total_after: Money<USD>
}
lemma transfer_preserves_total(
  before: Accounts, after: Accounts, amount: Money<USD>,
) -> Proof<after.total() == before.total()> {
  ...
}
```

Listing 28: Erased specification state and lemma.

6.5 Proof Modes

Mode	Behavior
check	Type, effect, ownership, exhaustiveness, and lightweight contracts; target policy may retain checks or warnings.
verify	Every declared proof obligation is discharged or explicitly assumed.
verified profile	No runtime fallback, panic, unchecked arithmetic, unverified unsafe code, or unbounded nondeterminism.
audit	Assumption ledger, unsafe inventory, proof coverage, solver provenance, and dependency-contract report.

These proof modes are target design, not current selectable verification profiles. Current `sol check` performs static checking and builds templates; `sol run` and `sol test` execute supported callable contracts under `CHECK`. Neither constitutes SMT proof discharge.

Evidence label	Meaning and boundary
Declared	Source states intent or a predicate; typed acceptance alone does not prove it.
Runtime-checked	A supported predicate was evaluated in a particular execution under the selected policy. Tests cover tested cases, not all inputs.
Proved-under-assumptions	A proof establishes a property only inside an explicit model and trusted assumptions. The bootstrap has no solver discharge; an obligation alone does not qualify.
Unknown	Unresolved, unsupported, omitted, or outside the checked/proved boundary. Unknown is not success or an empty dependency set.

6.6 Runtime Contract Fallback

External predicates may lie outside the solver fragment. `validate` retains a runtime check and yields a refined value; `assume` requires unsafe/proof-policy exception. Public APIs state static proof, dynamic validation, or boundary trust.

```
let email: EmailAddress = validate EmailAddress(input)
  .or_error(SignupError.invalid_email)?
let packet_length: PacketLength = unsafe assume_refined(raw_length)
```

Listing 29: Dynamic validation versus explicit trust.

Callable runtime checks and direct refined construction are implemented in the reference interpreter, and P2.6 closes bounded predicate bodies for later backend input. The proposed recoverable `validate` API, safe base projection, `assume_refined`, and proof-directed fallback above are not implemented. Direct construction can report a runtime refinement failure; it is not a `Result`-returning validation API.

6.7 Specifications and Generated Tests

The bootstrap implements only authored Boolean units:

```
test "empty input is rejected" {
  validate_input("") == false
}
```

Listing 30: Authored bootstrap unit tests.

Top-level `test "label"` expression declarations are private, zero-parameter, nongeneric, non-importable, and non-addressable. The body is contextually `Bool`; true passes, false fails, and a runtime error fails. Tests have no annotations, modifiers, contracts, authority clauses, or explicit effect clauses; their effects infer like private functions.

IMPLEMENTED `sol test [--diagnostic-format=human|json]` compiles a file or directory package exactly as `sol check`, renders compilation diagnostics while package source remains alive, then frees all frontend/package state. It discovers dedicated owning-IR TEST definitions and exact callable IDs in deterministic package-path then declaration order. Each test receives fresh default interpreter limits, checked transitive contracts, no host callback, and no ambient capability. Execution continues through all false and runtime failures and emits no timing. Exit status is zero exactly when compilation and every test pass, one otherwise, and two for usage. Successful JSON uses schema `sol.test-results`, version 1, with ordered entries containing path, decoded label, status (`passed`, `false`, or `runtime_error`), and nullable runtime diagnostic, plus total/passed/failed summary counts. Compilation failures retain `sol check` diagnostic JSON. Load and infrastructure errors use the versioned `sol.cli-error/1` object. JSON serializers preserve valid UTF-8 semantics by emitting valid sequences unchanged, escape syntax and controls normally, and render each invalid byte as a deterministic `\u00XX` Unicode-scalar marker; invalid bytes are not claimed to round-trip through JSON scalar semantics. Human labels display valid UTF-8 and safely escape invalid bytes. Human runtime failures report both declaration identity and the owning diagnostic file/aggregate offset.

FUTURE WORK `spec` examples, authored and generated properties, boundary generation, shrinking, seed control, and ghost state remain roadmap work. `spec` blocks will compile those forms into tests and obligations. Boundary values derive from refinements and mutation operators; generated coverage is reported separately.

```
spec normalize_username {
  example "trims and folds case" {
    input = " Alice.SMITH "
    output = "alice.smith"
  }
  property "idempotent" for all value: Text {
    normalize_username(normalize_username(value)) == normalize_username(value)
  }
  property "no surrounding whitespace" for all value: Text {
    let normalized = normalize_username(value)
    normalized == normalized.trim()
  }
}
```

Listing 31: Executable examples and universal properties.

6.8 Solver Discipline

SMT automation is constrained through typed theories, advanced-only triggers, deterministic budgets, and diagnostics naming expensive obligations. Builds record solver version/options. Timeout-sensitive heuristic proofs are unstable, never silently accepted.

- Quantifier-free arithmetic, constructors, equality, and finite maps form the preferred automatic fragment.
- Nonlinear arithmetic, unrestricted quantification, floating equivalence, and concurrency invariants may need lemmas or specialized engines.
- Obligations are named/addressable; suppression uses identity, not source line.
- Packages can bound solver time and memory per obligation and aggregate build.

All SMT and proof discharge remains future work.

6.9 Cost and Resource Contracts

Complexity and resources are specifications. Big-O claims can be checked against approved combinators/recurrences; hard byte/time limits combine static analysis and target certification. Weakened bounds are API changes.

```
function binary_search<T: Ordered>(values: SortedView<T>, target: T) -> Option<Index>
effects { pure }
cost {
  time <= logarithmic(values.length)
  allocation == 0 B
}
```

Listing 32: Complexity and allocation contract.

7 Concurrency, Protocols, Transactions, and Durability

7.1 Structured Concurrency

Tasks are lexically scoped. A parent cannot complete while children remain live unless ownership transfers to a supervisor. Errors and cancellation propagate by declared policy. Safe application code has no detached fire-and-forget task.

```
function load_dashboard(
  user: UserId,
  profile_service: capability ProfileService,
  message_service: capability MessageService,
  alert_service: capability AlertService,
  tasks: capability TaskScope,
) -> Result<Dashboard, DashboardError>
effects {
  network.call<profile_service>
  network.call<message_service>
  network.call<alert_service>
  task.spawn<tasks>
  task.suspend<tasks>
} {
  concurrent cancel_on_error {
    profile = profile_service.load(user)
    messages = message_service.recent(user)
    alerts = alert_service.active(user)
  }
  return Dashboard {
    profile = profile?
    messages = messages.or_default([])
    alerts = alerts?
  }
}
```

Listing 33: Structured concurrent fan-out.

A `concurrent` block creates a task scope and binds result handles. Policies include `cancel_on_error`, `collect_all`, and `supervise`. All results and resources must be consumed before exit.

This sketch assumes a supplied task-scope provider and explicit service capabilities. Task authority, suspension effects, cancellation, and ownership across suspension still need design decisions; the listing does not add an intrinsic or ambient effect to the bootstrap.

7.2 Race Safety

Ownership prevents unsynchronized shared mutation. Task transfers require `Send`; shared immutable data requires `Share`. Synchronization wrappers expose locking effects, poisoning, fairness, and cancellation. Verified profiles may check lock order and static cycles.

7.3 Actors and Channels

Actors isolate mutable state with typed asynchronous handlers. Channels are linear endpoints whose protocol can be a state machine. Mailbox capacity and overflow policy are part of actor type because they affect reliability and bounds.

```
actor InventoryActor(capacity = 1024, overflow = reject) {
  state inventory: Map<ItemId, Stock>
  message reserve(request: ReserveRequest)
    -> Result<Reservation, ReservationError> {
    ...
  }
}
```

Listing 34: Typed actor with explicit mailbox policy.

7.4 Protocol Types and State Machines

Protocols define state-indexed resources and transitions. A transition consumes one state and returns another, rejecting invalid sequencing. Explicit boundaries can erase dynamic protocol state to runtime tags.

```
protocol Connection {
  state disconnected
  state connected(socket: Socket)
  state authenticated(socket: Socket, session: Session)
  transition connect(from: disconnected, address: Address)
    -> Result<connected, ConnectError>
  transition authenticate(from: connected, credentials: Credentials)
    -> Result<authenticated, AuthError>
  transition query(from: authenticated, request: Query)
    -> Result<(authenticated, Response), QueryError>
  transition close(from: connected | authenticated) -> disconnected
}
```

Listing 35: State-indexed connection protocol.

7.5 Transactions

transaction is a language-level boundary backed by provider capability. It guarantees consistent commit/rollback, resource closure, and effect restrictions. Isolation, retry, and event semantics are explicit because databases differ.

```
public transaction transfer_funds(
  database: capability AccountsDb,
  clock: capability Clock,
  source_id: AccountId,
  destination_id: AccountId,
  amount: Money<USD>,
) -> Result<TransferReceipt, TransferError>
isolation = serializable
retry = safe_if_conflict
effects { database.transaction<database> clock.read<clock> } {
  ...
}
```

Listing 36: Transaction policy as part of the declaration.

A transaction may emit commit-coupled outbox events. Direct external network calls are rejected by default because rollback cannot undo them. Cross-system work uses sagas or durable workflows.

7.6 Durable Workflows and Sagas

A workflow models restart-surviving steps. Inputs, outputs, continuation state, retries, and compensation are serializable/versioned. Workflow code is deterministic relative to recorded events; direct clock, randomness, and network become durable commands.

```
workflow fulfill_order(order: OrderId) -> FulfillmentResult {
  let payment = step authorize_payment(order)
  retry exponential(max = 5)
  compensate refund_payment(payment.authorization)
  let shipment = step create_shipment(order)
  retry exponential(max = 8)
  compensate cancel_shipment(shipment.id)
  return FulfillmentResult { payment, shipment }
}
```

Listing 37: Durable saga with compensation.

7.7 Atomicity and Concurrency Verification

Advanced verified code can open named invariants in atomic sections and manipulate ghost state. Everyday code uses synchronization/protocol abstractions; the proof sublanguage isolates separation-logic reasoning for custom lock-free structures.

FUTURE WORK All concurrency, protocols, transactions, workflows, ownership-based race safety, and their verification semantics in this section remain target design.

8 Modules, Schemas, Interoperability, and Runtime Model

8.1 Packages and Editions

A package contains a manifest, module tree, features, target profiles, dependency locks, and policy. Editions stabilize syntax/core semantics. A new edition may change defaults or reserve keywords but does not reinterpret older packages.

```
package "inventory-service" {
  edition = 2027
  version = "0.4.0"
  targets = [native, wasm_component]
  profile.release = verified
  dependency "sol-http" = "^0.3"
  dependency "postgres" = "^1.1"
  capabilities {
    allow network.call<PaymentGateway>
    allow database.transaction<InventoryDb>
    deny process.spawn
  }
}
```

Listing 38: Package manifest sketch.

8.2 Stable Semantic Identities

Every exported declaration receives a stable semantic ID, derived by default from package, module, kind, and evolution token. An annotation retains identity across movement/rename.

```
@stable("billing.transfer.execute.v1")
public function transfer(...) -> ... { ... }
```

Listing 39: Stable declaration identity.

In the full target, stable IDs would anchor patches, API history, proof caches, documentation links, telemetry schemas, and deprecations, with incompatible-reuse checks. The bootstrap currently validates identity collisions, not general compatibility of reuse.

IMPLEMENTED The bootstrap assigns every top-level declaration a versioned 128-bit ID separate from its dense session handle. The default hashes normalized module path, declaration kind, and name, so source ordering and file placement do not affect it. A named public declaration may instead retain a validated package-local `@stable` token across module movement and rename. Collisions and malformed/private/implementation uses are rejected. HIR occurrence records cover declarations, imports, resolved expression/type names, implementation trait heads, and trait bounds. Selected inspection schemas expose these IDs with their own versioning contract. Manifests and dependency-qualified identities, member/local identity, incompatible-reuse analysis beyond collisions, and a full public serialized IR remain future work.

8.3 Application Entrypoint

An application entrypoint is marked by the argumentless `@entry` annotation. Ordinary compilation accepts zero entrypoints so libraries and tests remain valid, and accepts at most one across the complete package. An application command requires the compiled package to contain exactly one.

The marked declaration must be a named public top-level free function with a body, no type parameters, no effect-row parameter, and an explicit closed effects clause. Every parameter must use owned access and an unqualified, nongeneric root capability type. Ordinary values, callbacks, tuples, `borrow`, `inout`, and member receivers are not application inputs. The exact result type is either `()` or `Int64`.

A successful `()` result maps to process status 0. A successful `Int64` result maps to the same status only when it is in the inclusive range 0 through 255; values outside that range are application-boundary errors rather than truncating or wrapping. Panic, host failure, contract failure, resource exhaustion, and every other interpreter failure remain structured runtime failures and are never interpreted as returned status values. `sol run` reports those diagnostics and uses driver-failure status 1. Application

execution uses the bounded interpreter limits, with the existing deterministic defaults unless the host profile supplies explicit limits.

```
@entry
public function launch(console: capability Console) -> Int64
effects { console.write<console> } {
    console.write("hello")
    return 0
}
```

Listing 40: Explicit application entrypoint.

IMPLEMENTED The parser accepts argumentless `@entry` and rejects arguments or duplicates on one declaration. Package HIR rejects non-function, private, bodyless, generic, open-effect, and package-duplicate entrypoints. Type checking admits only owned nongeneric root-capability parameters and exact `()` or `Int64` results. Canonical owning IR retains the marker and independently rejects forged duplicate or malformed entrypoint metadata. The opaque validated-IR API resolves the entrypoint and maps only successful correctly typed results to exact process status. The E3/E4 host and `sol run` boundaries provide trusted capability injection, missing-entry application diagnostics, and runtime rendering; ordinary library compilation intentionally does not diagnose absence.

8.4 Trusted Interpreter Host Profile

Hosted application execution uses an explicit registry bound to one immutable validated-IR handle. Each entrypoint parameter is bound by ordinal to a fresh registry-owned root; callers cannot provide, observe, compare, or reuse root tokens. An operation grant is exact for one root and one uniquely named member of that root's capability. Before entering Sol code, execution rejects a missing root, a grant outside the entrypoint's closed effect row, a missing grant for a host-backed entrypoint effect, an ambiguous effect family, or a registry from another validated handle.

Safe host callbacks receive only borrowed ordered data arguments, an output value view, an interpreter-owned bounded failure buffer, and their registered context. They receive no `SolIr`, callable ID, root token, or private capability source. Host-backed members must be bodyless, closed, nongeneric, use one `Self`-rooted effect atom, have no `inout` parameter or result authority, and recursively use only `Int64`, `Bool`, `Text`, `()`, `Option`, and `Result`. Source-bodied capability members and lexical handlers remain ordinary Sol execution and need no host grant. A runtime operation without an exact root/member grant fails as an unbound operation rather than falling through to a broad callback.

The minimal E3 conventions are:

- `Console.write(Text) -> () effects { console.write<Self> }` for exact output bytes; no console input is provided.
- `Arguments.count() -> Int64 effects { process.arguments.count<Self> }` and `Arguments.get(Int64) -> Option<Text> effects { process.arguments.get<Self> }` over an immutable host-supplied argument snapshot. Index zero is the first application argument, not an executable path.
- `Configuration.read(Text) -> Option<Text> effects { configuration.read<Self> }` over an immutable deterministic host-supplied key/value snapshot. Missing keys return `none`; no ambient process environment is consulted.

Argument/configuration text returned by callbacks is validated, cloned, and charged to the interpreter's value-node and text-byte budgets. Every attempted permitted host dispatch is charged to the host-call budget before callback invocation; a zero host-call limit invokes no host code. The profile accepts at most 64 entrypoint roots, 256 outward effects, and 256 candidate capability members/grants, bounding preflight independently of package size. Steps, call depth, value nodes, text bytes, and host calls retain the deterministic interpreter defaults when all limits are zero and accept explicit per-dimension limits otherwise. Host failure bytes are written into interpreter-owned bounded storage and copied into the owned structured runtime diagnostic. Filesystem, network, clock, randomness, console input, live environment access, and process mutation are absent from this profile.

IMPLEMENTED The opaque `SolHostRegistry` API creates exact root bindings and root/member grants from validated entrypoint metadata. `sol_validated_ir_interpret_entrypoint` reads only contract policy and interpreter limits from its request, preflights authority, constructs capability arguments privately, and executes through translation-unit-private unchecked interpretation. The raw `SolInterpreterHostOperation` and raw authorization hook remain trusted mutable-IR interfaces and are still rejected by ordinary opaque-handle interpretation. Tests cover console, arguments, configuration, duplicate capability types with root-specific callbacks, missing roots and grants before execution, duplicate grants, cross-handle registries, host failure ownership, host-call limits, and malformed derived-capability entrypoints.

8.5 Interpreter Application Command

The bounded application command is:

```
sol run [--diagnostic-format=human|json] [--config=KEY=VALUE]
      <file.sol|package-directory> [-- arguments...]
```

Exactly one source/package operand is required. Application arguments begin only after `--`; index zero is the first following argument. At most 256 arguments and 1 MiB of aggregate argument bytes are accepted. Configuration uses repeated `--config=KEY=VALUE`, splits at the first `=`, rejects empty or duplicate keys, accepts empty values, and is bounded to 256 entries and 1 MiB of aggregate key/value bytes. Neither arguments nor configuration include implicit process, executable, source-path, or environment values.

After successful compilation, `run` transfers the opaque owning IR and destroys all frontend/package state before resolving and executing the entrypoint. Every capability parameter receives a root, but the command grants only exact standard declarations: capability, member, effect, parameter access/type, and result type must all match the Console, Arguments, or Configuration profile. Unsupported and lookalike host operations therefore fail preflight before Sol code or console output. Runtime contracts use the CHECK policy.

Human mode sends exact `Console.write` bytes to stdout with no added newline and sends application/runtime diagnostics to stderr. JSON mode captures at most 1 MiB of console bytes and emits exactly one post-compilation object with schema `sol.run-result`, version 1, outcome returned, `application_boundary_error`, or `runtime_error`, nullable `exit_status`, base64 console data, and a nullable symbolic diagnostic. Compilation diagnostics retain `sol.diagnostic/1`; load/infrastructure failures retain `sol.cli-error/1`.

Successful `()` returns status 0 and successful in-range `Int64` returns the identical status from 0 through 255. Missing entrypoints (`SOL-RUN-001`), out-of-range results (`SOL-RUN-002`), profile/preflight failures (`SOL-RUN-003`), and runtime failures return driver status 1; invalid command usage returns 2. Application results of 1 or 2 intentionally overlap driver statuses, so diagnostics or the JSON outcome distinguish them. Output transport failure overrides every otherwise returned application status with driver status 1.

IMPLEMENTED `sol run` uses `SolCompilationSession`, `SolValidatedIr`, exact standard-profile discovery, `SolHostRegistry`, and hosted entrypoint interpretation without raw IR access. CLI tests cover file/package execution, Unit and signed status boundaries, missing/duplicate entry behavior, exact console and base64 output including padding, arguments and configuration, partial operation grants, unsupported/lookalike authority before output, panic diagnostics, JSON phase separation, usage errors, and closed-output transport failures.

8.6 Canonical Typed Interpreter IR

Successful bootstrap compilation lowers to one owning, self-contained typed IR that is the sole semantic input to the compiler-internal reference interpreter. A shared single-use compilation session owns file, directory, NUL-terminated text, and length-delimited in-memory package construction; exact phase order; diagnostics; private package/HIR/type/effect/contract arenas; and reverse-order teardown. Callers receive copied scalar summaries and diagnostic records, session-owned diagnostic and inspection rendering, and borrowed strings valid only until IR transfer or session destruction. A successful session transfers exactly one opaque validated-IR handle without allocation; narrow read-only definition/path projections, effects rendering, and interpretation do not expose the internal `SolIr`, and the handle remains valid after session destruction. The IR has independent structural-type, declaration, callable/member, local, field, variant, expression, recursive-pattern, statement-list ID, match-arm-list ID, operand, dispatch-evidence, effect, source-file, obligation, and snapshot arenas; it does not embed or retain frontend syntax, HIR, type, effect, or contract tables. Match arms retain an optional guard and exact binding/cleanup ownership. It owns compact aggregate source bytes plus per-file paths/ranges, decodes integer and the supported `\n`, `\r`, `\t`, `\\`, and `\\` string escapes, orders call and constructor operands by resolved formal or field identity, represents payloadless variants directly, classifies executable calls and bound operations, retains receivers/private wrapper sources/result authority, records concrete and forwarded generic trait evidence, and points contracts at IR expression and snapshot IDs. Effect atoms own copied names and canonical local/self authority references and are sorted by those semantic values.

IMPLEMENTED The bootstrap exposes the opaque `SolCompilationSession` and `SolValidatedIr` API with distinct success, source rejection, load failure, resource failure, and invalid-argument outcomes. The session is single-use; invalid compile or transfer transitions do not rerun phases, and transfer destroys all frontend state before returning its allocation-free immutable IR owner. The lower-level bootstrap also exposes `sol_ir_init`, `sol_ir_free`, `sol_ir_lower`,

`sol_ir_lower_scoped`, and `sol_ir_validate` as C compiler-internal APIs. Lowering requires a bytewise-empty output, validates frontend pointers, exact counts, nested type/effect/contract metadata, spans, links, owners, and slices before use, fails transactionally with a structured internal diagnostic, and validates every retained executable node and IR-native cross-reference. Definition-to-callable links and callable-to-definition ownership are exact and kind-matched; ordinary definitions cannot alias TEST entries. Capability, trait, and implementation member callables occur exactly once in their matching owner slice, with no foreign, duplicate, shared, or orphan member-table entries. Bidirectional expected-type flow supports nested built-in `ok`, `err`, `some`, and `none` calls in returns, explicitly typed call/method arguments, distinct and record/variant fields, equality, propagation operands, terminal block values, and context-typed branches; ambiguous constructor uses are rejected. Propagation is limited to exact `Option` and `Result`, records its kind and `Result` residual type, and rejects user generic applications or mismatched errors. Generic inference is deliberately left-to-right: a contextual builtin is supported once earlier operands determine every required type argument; a builtin that precedes the needed evidence is diagnosed rather than inferred by global backtracking. Builtin and type-application heads are non-executable and accepted only as parts of supported call, constructor, record, or member chains; first-class heads and explicit builtin type applications are rejected. Unknown string escapes, static-authority effect atoms, and frontend constructs lacking explicit bounded lowering metadata are rejected. This IR is explicitly unstable and promises neither serialization nor public ABI compatibility.

IMPLEMENTED Compilation sessions accept explicit limits, with deterministic defaults, for source-file and aggregate package bytes, source files, directory depth and visited entries, tokens, persistent compiler-arena entries, diagnostics, cumulative allocation bytes, and allocation requests. One session-scoped meter covers package construction and every compiler phase; limit failures produce `SOL_COMPILATION_RESOURCE_FAILED` with the first exhausted resource and publish no validated handle. Directory traversal uses verified live descriptors plus `fstatat` and `openat`; source descriptors reject symbolic links and non-regular files, match the identity discovered by the parent, read the measured extent plus an EOF probe, and verify identity, size, and nanosecond modification/change timestamps after reading. Duplicate source identities are rejected.

IMPLEMENTED Statically known generic-call recursion uses sparse outgoing/incoming adjacency and iterative strongly connected components rather than a dense transitive-closure matrix. Diagnostics remain source-definition ordered. Effect inference packs ascending definition members by existing SCC and iterates only the active component while preserving its monotonic deterministic fixed point. The opaque validated-IR handle is the one public validation certificate: validated interpretation and effects rendering share translation-unit-private unchecked implementations after successful lowering, while public raw mutable-IR operations continue complete structural and ownership validation on every call. Checked-in warning-clean normal/sanitizer CI, optional Clang/libFuzzer parser/package/scalar-IR harnesses, and deterministic hashed generic, repeated-test, and package stress workloads form the initial release baseline.

IMPLEMENTED `sol_ir_analyze_ownership` runs over completed typed IR and annotates every executable local read as copy, move, or direct receiver use. Lowering publishes no IR if ownership fails. `sol_ir_validate_ownership`, called by ordinary IR validation, recomputes control-flow and structural-copy facts rather than trusting annotations. Whole-local state is joined across branches, matches, and short-circuit expressions, excluding terminated paths.

IMPLEMENTED Canonical IR retains structured region label/body/span nesting and exact cleanup slices in successful source order for every executable block's `let` locals and every match arm's pattern locals. Independent validation rejects invalid slices/IDs, duplicate, foreign, wrong-kind, wrong-order, or borrowed cleanup owners, cleanup mismatches, shared executable statements, and malformed region bodies before execution. Lowering remains transactional. These facts are compiler-internal and do not independently add an external inspection expression, type, or local kind.

IMPLEMENTED Successful syntax trees validate every arena count/pointer pair, top-level and nested discriminant before union access, active payload span, and linked-arena owner. Type arguments, parameters, fields, variants, effects, members, methods, patterns, statements, expressions, contracts, and their lists reject cycles, sharing, and orphans. Owning IR independently checks negative/out-of-range semantic enums; canonical type shapes; generic-aware literal, operator, call, constructor, field, branch, match, block, propagation, and handler type relations; callable body/return context; exact current-callable provenance roots; and malformed spans/owners before interpretation. Source fixtures cover every

successful AST expression kind and every owning-IR expression, statement, and pattern kind; composite second-file package fixtures exercise populated expression and linked arenas at nonzero relocation offsets, and inspection/formatter/interpreter preflights have focused category coverage.

IMPLEMENTED Canonical owning-IR place and place-projection arenas encode one local or computed-value root followed by one contiguous ordered projection slice. Local and nested/computed field reads use the same representation; field chains are maximally flattened, and each root/projection retains its source span and exact instantiated intermediate type. Validation proves count/pointer, discriminant, inactive-ID, field-owner, generic-substitution, final-type, unique place-expression owner, and unique projection-slice owner invariants. Ownership validation independently recomputes prefix-overlap loans, unavailable affine path frontiers, exact reinitialization, writable exclusive roots, region-safe writeback, and conservative projected-authority restrictions. Runtime aggregates may contain internal moved-field holes; strict-ancestor reads fail, exact replacement restores the slot, and cleanup skips holes. Projected reads and updates are step-metered. Source-call `inout` results are staged, every receiver/operand destination is preflighted, then all writebacks commit. Index and dereference projection discriminants remain reserved and rejected. These compiler-internal facts do not independently change external inspection.

8.6.1 Compiler-Internal Reference Interpreter

IMPLEMENTED The C17 library API in `include/sol/interpreter.h` deterministically evaluates validated owning IR after every frontend object has been freed. Sparse frames bind global IR local IDs, receivers, derived capability sources, generic substitutions, and invocation evidence. Owned runtime values cover primitives, immutable records/enums, `Option`, `Result`, distinct/refined values, capabilities, functions, and bound operations. Execution includes checked arithmetic, structural equality, control flow, propagation, direct and indirect calls, explicit trait dispatch, root-preserving wrappers, host operations, exact handlers, and runtime contracts. Dedicated TEST callables require an explicit request. Host callback results are borrowed views that are validated and deep-cloned before interpreter temporaries are released. External capabilities require their canonical private-source chain and one preserved root. Steps, call depth, value nodes, text bytes, and host calls are independently bounded; failures use stable structured codes and owned source-file paths. Named operands execute in canonical formal order because source operand order is not retained. IGNORE and CHECK are supported callable-contract policies; refined construction validates independently of that policy. This remains an interpreter profile, not a VM, bytecode engine, code generator, or production runtime.

Block lets and match bindings unbind/free in reverse metadata order on normal, return, propagation, and runtime-error flow. Callable parameters and root locals clean in reverse successful binding order; moved slots are skipped and outgoing flow values survive cleanup. Apart from its explicit observation hook, cleanup executes no Sol code or host operation and cannot replace diagnostics. The unstable synchronous/infallible observer reports local ID plus monotonic ordinal through `result.cleanup_actions`, only after IR and complete argument validation. Cleanup has no execution limit because a started unwind must finish.

IMPLEMENTED Raw `sol_interpret`, its raw-IR host callback, and its raw authorization hook remain trusted compiler-test interfaces. Ordinary opaque validated-handle interpretation rejects them with `SOL_INTERPRETER_INVALID_REQUEST` before execution, so no private IR escapes. Hosted entrypoint execution instead uses the exact opaque registry and data-only callback profile above.

IMPLEMENTED Raw host-operation failures write into interpreter-owned fixed-capacity text storage with an authoritative length. Empty failures select a stable default; embedded NULs and oversized lengths are rejected, and diagnostics copy at most the declared bytes without scanning borrowed storage. Successful borrowed host values must remain valid through the enclosing interpretation call.

8.7 Schema Definitions and Evolution

Persistent types carry versions and compatibility policy. Migrations are typed total functions whose loss behavior can be verified. Encoders retain unknown fields where wire formats allow, preventing destructive rolling-upgrade read/modify/write.

```

record UserProfile version 3 compatible backward {
  id: UserId @field(1)
  display_name: Text @field(2)
  locale: Locale = Locale.en_us @field(3)
  preferences: Preferences @field(4)
}
migrate UserProfile from version 2 to version 3 {
  locale = Locale.en_us
  preferences = Preferences.default()
}

```

Listing 41: Versioned schema and migration.

Field identities are independent of source ordering. Removal/change creates compatibility obligations. Database, event, and file schemas share a model but use distinct deployment adapters.

8.8 API Compatibility

The compiler computes a public semantic fingerprint over types, effects, errors, contracts, traits, layout, and protocol states. Tools classify source, binary, wire, behavior-strengthening, behavior-weakening, and breaking changes; SemVer recommendations remain owner decisions.

8.9 Foreign Function Interface

The C ABI is the universal native boundary. Headers and safe wrappers cover representable types; algebraic values use generated handles/interface records. FFI declarations include ownership, nullability, threading, blocking, errors, and effects.

```

extern "C" function zlib_compress(
  input: foreign_slice<Byte>,
  output: foreign_mut_slice<Byte>,
) -> CInt
ffi {
  ownership input = borrowed
  ownership output = borrowed_exclusive
  nullability = forbidden
  thread = any
  blocking = false
}
effects { ffi.call<zlib> unsafe }

```

Listing 42: Explicit FFI contract.

8.10 WebAssembly Components

The longer-term Component Model proposal would provide interface-driven composition and capability-oriented hosting, mapping records, enums, results, options, resources, and worlds to canonical interfaces. The first production track instead targets core Wasm over the frozen E6 profile, with a bounded host adapter. Neither backend nor Component Model integration exists at this baseline; effects disclose requirements, while explicit host grants supply authority.

8.11 Runtime Profiles

The following profiles are long-term candidates, not implemented runtime modes or the order of delivery. The reference interpreter is retained as a semantic reference, not a new VM commitment; the initial production target is Wasm.

Profile	Runtime model
Native static	AOT executable/library, deterministic destruction, configured allocator, optional async runtime.
Native hosted	AOT plus reflection metadata, loader, tracing, and supervised actors.
WebAssembly component	No ambient host access; imports define capabilities/resources.
Embedded	No OS assumptions, selectable allocator, interrupt-safe subset, heap-free option.
Sol VM	Portable bytecode for REPL, tests, migrations, and fast cycles; not performance reference.

8.12 ABI and Layout

Internal P2 layout is experimental and unstable. The target `@repr(C)`, `@repr(transparent)`, and stable ABI annotations would opt into guarantees; these are not current source features. Future public native ABI packages would freeze layout, calling convention, panic policy, and allocator ownership and emit an ABI manifest.

FUTURE WORK Package manifests/dependencies and dependency-qualified identity, schema evolution tooling, compatibility reports, FFI, Wasm emission, production runtime profiles/ABI, and ABI manifests remain absent. Executable owning IR, package-local stable top-level IDs, selected versioned inspection schemas, and experimental P2 representation/layout are already implemented; internal layouts do not establish a public ABI.

9 AI-Native Development and Semantic Editing

9.1 Design Objective

AI assistance is a consumer of compiler APIs, not a privileged bypass. A model proposes intent, code, proof, or patches; the compiler decides semantic validity. IDEs, refactoring tools, CI, and human automation use the same interfaces.

Text remains human-readable source of record, but edits can target declarations, parameters, effects, control-flow nodes, contracts, and schema fields by stable identity. Formatting materializes canonical source.

TARGET DESIGN The full intent/patch/graph workflow below is a long-term proposal. The smaller M experiment starts with selected existing inspection projections, source excerpts, ordinary edits, interpreter checks, and human approval, without waiting for full public IR or SMT. No M feature is built by this refresh. Stable top-level IDs have package-local scope; dense IDs are snapshot-local and inspection bytes are not guaranteed semantic cache keys. Unchanged signatures/effects do not establish unchanged behavior.

9.2 Intent Blocks

Intent records purpose, preservation requirements, forbidden outcomes, priorities, and assumptions. Free prose explains; enforceable fields map to contracts, lints, or review rules. Intent does not become magically executable.

```
intent transfer {
  purpose: "Move funds atomically between two accounts."
  preserve:
    - total money across both accounts
    - account identity
    - unrelated account fields
  forbid:
    - partial transfer
    - negative balance
    - transfer to the same account
  priority: correctness > observability > performance
}
```

Listing 43: Structured intent metadata.

9.3 Semantic Patch Language

The patch language operates on the typed semantic graph. It can add/remove parameters, alter effects, insert at semantic anchors, update variants, retain identity through rename, and attach preservation constraints. Ambiguous or stale targets are rejected.

```
patch billing.transfer::transfer {
  require base_fingerprint = "sha256:9e4d..."
  add parameter memo: Option<Text> after amount
  replace effect database.write<Accounts>
    with database.transaction<Accounts>
  insert before event FundsTransferred {
    audit.record_transfer_attempt(
      source = source_id, destination = destination_id, amount,
    )
  }
  preserve {
    all existing error variants
    atomicity
    behavior except optional memo support
  }
}
```

Listing 44: Semantic patch with stale-base protection.

9.4 Machine-Readable Diagnostics

Every diagnostic has a stable code, semantic subject, spans, violated rule, related declarations, proof context, and bounded repair candidates. Human prose presents this structure; repairs are constrained classes, not automatically applied guesses.

```
diagnostic {
  code: "SOL-EFFECT-014"
  severity: error
  subject: "billing.transfer.transfer"
  violation: {
    attempted_effect: "network.call<AuditService>"
    declared_effects: ["database.transaction<Accounts>", "clock.read"]
  }
  valid_repairs: [
    add_effect("network.call<AuditService>"),
    inject_capability("AuditSink"),
    move_operation_to_caller,
  ]
}
```

Listing 45: Structured effect diagnostic.

The bootstrap already emits structured human/JSON diagnostics with stable code strings and spans. Stable cross-version subject IDs and full repair schemas remain target design.

9.5 Change-Oriented Compilation

`sol check-change` compares semantic graphs and reports effects, weakened postconditions, stronger preconditions, errors, schemas, protocols, allocation, proof assumptions, and tests, not just text.

```
$ sol check-change HEAD~1..HEAD
Public behavior:
+ Transfer requests accept an optional memo.
Effects:
! billing.transfer.transfer now calls AuditService.
Contracts:
= Atomicity preserved.
= Existing errors preserved.
Cost:
! Worst-case network calls increased from 1 to 2.
Verification:
842 obligations proved
3 new obligations proved
1 obligation unresolved:
  audit failure cannot make a committed transfer appear failed
```

Listing 46: Behavior-change report.

9.6 Context Bundles

A tool requests a bounded bundle of declarations, signatures, intent, contracts, callers/callees, effect providers, protocols, diagnostics, and recent semantic changes.

- Bundles are deterministic and hash-addressed.
- Sensitive source may be replaced with interface summaries.
- Omitted context is recorded so unsupported assumptions are visible.
- Generated edits cite subjects/base hashes for safe replay or rejection.

9.7 Trust and Approval Model

Trust does not depend on human versus AI authorship. It derives from review policy, verified properties, tests, unsafe assumptions, provenance, and signers. Policy may require approval for new capabilities, weakened contracts, unsafe blocks, schema changes, or resource changes.

9.8 Canonical Intermediate Representation

The proposed full public semantic IR would be readable, serializable, and versioned, representing resolved identities, types, effects, contracts, regions, control flow, and obligations in compact binary or canonical text. Source and IR need not be

isomorphic: comments/presentation remain metadata, while inferred types/effects become explicit. This is not the compatibility contract of current inspection projections or internal owning IR/MIR.

```
function billing.transfer.transfer {
  stable_id "billing.transfer.execute.v1"
  input source: identity.AccountId
  input destination: identity.AccountId
  input amount: money.Money<USD> refined greater_than_zero
  output Result<TransferReceipt, TransferError>
  effects { database.transaction<Accounts> clock.read }
  obligations {
    precondition same_account_forbidden
    invariant total_balance_preserved
    postcondition receipt_matches_commit
  }
}
```

Listing 47: Human-readable Sol semantic IR sketch.

FUTURE WORK Intent declarations, patches, context bundles, change reports, policy gates, and full canonical serialized Sol IR are target design. Selected versioned syntax/HIR/type/effect/contract/diagnostic projections are implemented, but exclude raw owning IR, MIR, deserialization, complete fact coverage, and cache-key suitability. Small proposed tools can use these projections with explicit omissions rather than making the full graph a prerequisite.

10 Compiler Architecture, Toolchain, and Developer Experience

10.1 Compiler Pipeline

The target compiler consists of deterministic cacheable stages. Parsing produces lossless syntax; resolution and macro-free elaboration produce canonical AST; type/effect/ownership checking produce typed HIR; contract elaboration produces obligations; Sol IR lowers to ownership-explicit MIR and backends.

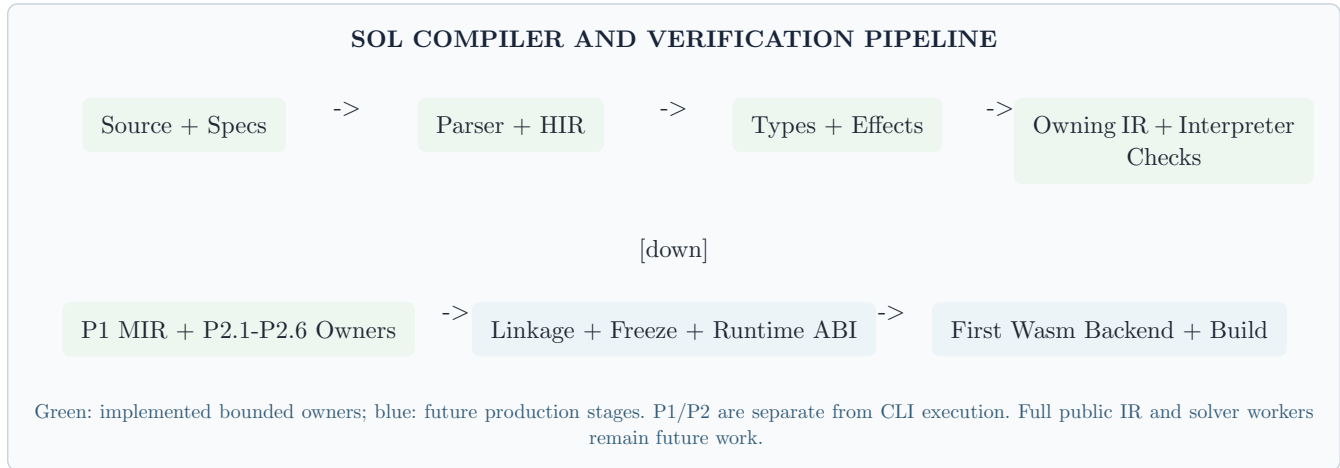


Figure 2: Implemented execution and experimental lowering, followed by the planned production boundary.

10.2 Front End

The target front end provides:

- Incremental error-recovering parsing with a lossless tree for IDE/formatter use.
- Edition-aware lexing/parsing and no user token macros.
- Deterministic name resolution and explicit import graph.
- Constraint-based type/effect/region inference bounded for predictable diagnostics.
- Exhaustiveness and protocol-state checking before proof generation.

The bootstrap implements lossless tokens, recovering core parsing, token-preserving idempotent formatting, arena-backed syntax, deterministic package-session definition IDs, lexical and explicit multi-file module/import resolution, declaration-owned type resolution, typed HIR foundations, generic and nongeneric records/enums/functions/distinct/refined declarations, bounded coherent traits and constrained calls, exact invariant applications, recursive patterns and pure match guards, closed effects/capabilities/handlers, executable callable contracts and refined construction, loop-obligation templates, whole-local affine ownership, callable-scoped projected shared/exclusive borrowing, statement-only loops and nested lexical regions, and deterministic internal cleanup. It does not yet implement incremental parsing, dependency packages or manifests, first-class borrows/regions, allocator regions, lifetime generics, user `Drop`/close protocols, static proof discharge or runtime loop-obligation validation, richer trait or row-polymorphic generics, protocols, or width/reordering formatter policies.

10.3 Verification Engine

The target verifier normalizes contracts into typed logical IR, partitions by theory, and dispatches to analyzers/isolated solver workers with deterministic budgets. Results include assumptions, counterexamples, and hashes. Counterexamples map to source values/protocol traces. Diagnostics explain relevant paths and theories rather than merely reporting timeout.

No verifier or solver integration exists today.

10.4 Sol IR and MIR

Sol IR is high-level, typed, and effectful for tools and transforms. MIR is SSA/control flow with explicit moves, borrows, drops, regions, panic policy, and target-independent layout. Proofs attach mainly to Sol IR; memory/optimization validation attaches to MIR.

An MLIR-inspired multilevel approach can preserve Sol operations until semantics have been used while lowering to LLVM, Cranelift, WebAssembly, or embedded targets. Adopting MLIR itself is optional; textual, in-memory, and serialized forms should carry the same semantics.

10.5 Backends

Backend	Role
WebAssembly	First planned production target, frozen E6 core and exact host adapter; no Component Model dependency. Toolchain selection/pinning remains open.
LLVM	Possible later optimizing native AOT and link-time optimization; not selected as the first delivered backend.
Cranelift	Possible later fast development/JIT option; not an active parallel backend track.
C	Possible later debug/integration output; the compiler's C17 implementation is not a C-emitting backend.
Sol VM	Deferred possible bytecode/REPL design, not the existing reference interpreter.

The current C17 compiler executes its bounded core through a reference interpreter and has experimental MIR/P2 owners, but emits no executable backend artifact. P3.W1 is an early timeboxed ABI experiment after relevant conventions are tested, not production backend completion.

10.6 Incremental Compilation

Incremental keys are semantic where possible. Parsing, resolution, interfaces, monomorphization, obligations, and code generation use separate caches. A private implementation change should not invalidate downstream checking if its exported fingerprint is unchanged.

10.7 Command-Line Tool

```
sol new service inventory
sol fmt
sol check
sol effects path/to/package
sol verify --profile critical
sol test path/to/package
sol run
sol build --target native --release
sol build --target wasm-component
sol check-change origin/main..HEAD
sol explain SOL-EFFECT-014
sol audit --unsafe --capabilities --proofs
sol patch apply change.solpatch
```

Listing 48: Unified target sol surface.

The target commands above are illustrative, not a supported CLI inventory or frozen target-option syntax. The implemented commands are `check`, `test`, `run`, `effects`, `inspect`, and `fmt`; `run` uses the reference interpreter with exact hosting and `CHECK` contracts. Native and component build spellings describe deferred ambitions, not the first Wasm artifact interface.

The bootstrap currently implements `sol check`, `sol test`, `sol run`, `sol effects`, `sol inspect`, and `sol fmt`. Testing accepts one file or recursively discovered directory package and executes authored Boolean units deterministically. Application execution uses the exact hosting and `CHECK` policy in Section 8. Formatting accepts one file or package; `--check` reports noncanonical files and `--stdout` formats one file without writing. Effect inspection compiles the same bounded package, then deterministically reports normalized callable rows, lexical authority, and source-ordered direct, capability, method, and callback call sites from owning typed IR. Dynamic callbacks and forwarded generic method evidence remain explicitly unresolved; this bounded command result is not a stable serialization of compiler schemas or internal IR.

`sol inspect` emits one compact outer `sol.inspection` version-3 object after successful compilation. Syntax and types are artifact version 3, while HIR, effects, contracts, and diagnostics remain version 1. Syntax v3 exposes recursive pattern and match-arm catalogs plus match ownership; types v3 exposes per-pattern types and variant, field, tuple-ordinal, and child relationships, backed by snapshot-local variant and field catalogs. Artifacts preserve original source bytes through base64, package-relative paths and file-local byte spans, package-local stable top-level semantic IDs, tagged facts, and explicitly

snapshot-local dense references. They exclude C layouts, pointers, capacities, native sentinels, enumeration numbers, and internal IR. Versions 1 and 2 remain historical; version 3's schema and documentation govern current compatibility. Failures retain the diagnostic and CLI-error JSON envelopes. Inspection construction failures write no partial successful envelope; output transport can truncate a constructed envelope and returns status 1.

10.8 Language Server and IDE

- Completion includes effects, capabilities, protocol states, and valid constructors.
- Editors show obligation status and counterexamples.
- Refactors are semantic patches previewed as behavior changes.
- Call hierarchy annotates effects and authority.
- Ownership views appear on request or inference failure.
- Generated tests/proof holes remain separate from production code.
- CI, IDE, and agents share a stable diagnostic protocol.

10.9 Build Reproducibility

Target manifests pin edition, package graph, target, proof engines, solver versions, build scripts, generated hashes, and capability policy. Build scripts run as WebAssembly components with declared authority rather than arbitrary host shells.

For this manual, `docs/specification.typ` is the authoritative editable source. The root `Sol_Programming_Language_Design_Specification_v0.2.pdf` is generated, never edited. With Typst 0.15.1, the offline reproducible command from the repository root is:

```
typst compile docs/specification.typ Sol_Programming_Language_Design_Specification_v0.2.pdf
```

No remote package or external asset is required. When CMake finds Typst, the optional `manual` target runs the same compiler command.

10.10 Documentation

API documentation derives from signatures, effects, errors, contracts, examples, protocols, schemas, and intent. It shows authority, suspension/allocation, recoverable failures, and proven versus runtime-checked properties.

11 Standard Library, Ecosystem, and Security

11.1 Standard Library Principles

The core library is small, stable, and freestanding. Hosted functions live in capability-oriented packages. APIs avoid hidden locale, clock, environment, filesystem, allocator, and executor dependencies. Algorithms expose complexity/allocation where practical.

11.2 Library Layers

Layer	Contents
core	Primitive traits, algebraic types, numeric operations, text/bytes views, ownership helpers, compile-time facilities.
alloc	Vectors, maps, strings, reference counting, arenas, allocator interfaces.
std	Filesystem, network, process, clock, random, concurrency, serialization, logging, platform abstractions.
verify	Propositions, proof combinators, model collections, solver theories, ghost helpers.
component	WebAssembly interfaces, resources, worlds, host adapters.
embedded	No-OS abstractions, volatile memory, interrupts, fixed-capacity collections, hardware capability traits.

11.3 Text, Time, and Randomness

Text distinguishes bytes, scalars, graphemes, normalization, and locale-sensitive behavior. Time distinguishes monotonic duration, civil time, instants, and timezone data. Randomness separates deterministic pseudo-random state from secure host entropy. These prevent convenience APIs from hiding data or nondeterminism.

11.4 Networking and Services

Network APIs are asynchronous and cancellation-aware by default. DNS, TLS roots, proxies, and credentials are capabilities. Generated service clients preserve typed errors, deadlines, idempotency, and retry rather than generic transport exceptions.

11.5 Logging, Metrics, and Tracing

Observability is an effect handled by no-op, buffered, test, or production sinks. Structured events use stable schema IDs. Pure functions cannot log; callers consume returned diagnostics or handle tracing around an effectful computation.

11.6 Package Registry and Supply Chain

- Content-addressed artifacts and reproducible source archives.
- Signed publishers and optional organization attestations.
- Machine-readable capability, unsafe, FFI, build-script, and proof metadata.
- Lockfiles pin dependency graphs and integrity hashes.
- Policy can deny new transitive capabilities or unsafe code.
- Releases publish semantic API diffs.
- Build scripts and procedural derives run in declared sandboxes.

11.7 Security Model

Sol combines memory safety, capability authority, visible effects, package sandboxing, schema validation, and auditability. Types do not eliminate logic vulnerabilities; they make authority and assumptions smaller and reviewable.

Threat	Language/tool response
Memory corruption	Safe ownership/borrowing; raw memory confined to audited unsafe and FFI.
Ambient authority	Capabilities for filesystem, network, process, environment, database, clock, randomness.
Dependency compromise	Signed/content-addressed packages, sandboxed builds, capability diffs, policy gates.

Threat	Language/tool response
Schema confusion	Nominal/field IDs, migrations, unknown-field policy, compatibility checks.
Unicode spoofing	Identifier normalization and public-API confusable detection.
AI-generated unsafe change	Policy gates on effects, unsafe, weakened contracts, and assumptions.
Denial of service	Resource contracts, bounded profiles, mailbox policy, timeout, cancellation.

11.8 Reflection and Serialization Safety

Runtime reflection is opt-in package metadata and cannot bypass invariants. Deserialization is a validation boundary with typed errors; refinements are validated or carry trusted schema proofs. Standard serialization excludes arbitrary object graphs and executable constructors.

FUTURE WORK The broader standard library, registry, supply-chain enforcement, sandboxed builds, reflection, and serialization are target design. Bootstrap affine ownership, lexical loans, effect/capability checking, deterministic resource limits, and exact hosted grants are implemented foundations, not a complete application security model. Permission does not prove per-request authorization, tenant isolation, or information-flow safety.

12 Implementation Roadmap, Risks, and Open Questions

12.1 Recommended Implementation Strategy

Begin with a deliberately small core proving interactions among canonical syntax, algebraic types, ownership, effects, contracts, semantic identities, and diagnostics. Simultaneously building full proof automation, workflows, native targets, and an ecosystem would obscure core coherence.

E1-E6 established the bounded frontend/interpreter application core, and P1 through P2.6 are complete at 6ec4ba9. Language breadth remains frozen. The live execution cursor is:

```
P2.7 symbols and whole-program linkage (next overall and production)
-> M1 first-user workload and experiment charter
-> P2.8 concrete-program freeze and census
-> P3 runtime ABI -> P4 Wasm backend/adaptor -> P5 build artifacts
```

P2 is still open. M1 publishes subsequent interleaving; if infeasible, it records the blocker and work continues to P2.8 without inventing prerequisites. After P2.8, the M2-M5 experiment may proceed independently alongside P3, subject to its own gates, staffing, and exclusive file ownership. One coordinator maintains the sole next-overall cursor. Closures, collections, user resources/allocators, unsafe, C FFI, manifests/dependencies, full public IR, concurrency, and broader handlers are not implicit P2-P5 requirements.

12.2 Executable Bootstrap Baseline

The September 9, 2026 snapshot of 6ec4ba9 provides the following. This is a documentation summary, not a fresh validation report; detailed compatibility boundaries are in compiler status.

- Lossless lexing, recovering parsing for core declarations/contracts/body expressions, arena syntax, deterministic package-session definition IDs, lexical and explicit multi-file module/import HIR resolution, and source-aware human/JSON diagnostics.
- Token-preserving syntax formatting with canonical whitespace, checked parse/token preservation, byte idempotence, and transactional package rewrites.
- Primitive types; exact variable-arity interned built-in/user/structural-tuple applications; bounded generic records, enums, and free functions; constructors; recursive patterns and pure guards; nested usefulness/exhaustiveness for `Bool`, records, tuples, and closed/open generic enums; expression/call/return checking.
- Nominal generic distinct declarations and explicit construction; refined declarations with representation-typed `self`, `Bool`/purity checking, obligation templates, and direct runtime predicate evaluation before nominal construction.
- Nongeneric coherent traits; exact closed implementations; one inline free-function bound; symbolic bound forwarding; immediate type-directed method calls; checked method-resolution metadata and exact closed method effects.
- Closed structural function types and bounded declaration-owned callback row parameters; exact function and bound-operation effects; callback subset checking; local and per-call row inference.
- Least-fixed-point recursive inference over call-graph SCCs, including parameter and `Self` substitution.
- Capability parameters, immutable authority aliases, normalized finite may-origin sets for computed `if/match`, and conservative root expansion.
- Disclosure-only rows with lexical capability authority for resource effects and a closed authority-free `panic/diverge` subset.
- Deterministic inspection of normalized callable rows, lexical authority, and statically represented call edges through `sol effects`.
- Exact authority-preserving returns and nominal checked single-source wrappers.
- Exact capability-backed handlers with syntax, source/provider matching, scoped root-sensitive subtraction, residual/provider effects, runtime metadata, and singleton target limitation.
- Structured contracts resolved in fresh signature scopes; generic template typing; `Bool` typing; finalized purity; `Result` outcome-specific `result`; distinct `old` snapshots; deterministic obligation templates and runtime callable `CHECK`, including approved hosted members. `sol run` and `sol test` enable `CHECK`.
- Versioned 128-bit top-level semantic IDs, package-local explicit evolution tokens, collision validation, and resolved declaration/import/expression/type/trait occurrence records.

- Structural bootstrap copies, path-sensitive affine moves, prefix-overlap projected loans, use-after-move/conflict/escape diagnostics, structured control-flow joins, validated access/local-use IR metadata, and ownership-aware interpretation.
- Statement-only nested lexical regions; exact block-let and match-binding cleanup metadata; affine region-escape diagnostics; and reverse deterministic interpreter storage cleanup on every flow.
- Initialized mutable bindings; fixed-type local/record-field assignment; UPDATE ownership metadata; exact replacement cleanup; moved-field reinitialization; and region-safe affine updates/writeback.
- Exhaustive current syntax/owning-IR discriminant and active-payload validation; linked-arena ownership/cycle/orphan rejection; generic-aware executable type and callable-context checks; and per-kind relocation/inspection/formatter/malformed-input fixtures.
- Canonical typed owning-IR places with local/computed roots, flattened field projections, exact intermediate types/spans, unique arena ownership, and reserved but non-operational index/dereference projections.
- Successful source-call copy-in/copy-out for writable local-rooted `inout` places, including callbacks, recursion, and implementation methods; host/top-level exclusive deferral remains explicit.
- Explicitly typed authority-free uninitialized locals; reachable-path definite-initialization joins; checked atomic `Int64` compound assignment; and lexical whole-local owned `modify` scopes.
- Statement-only `loop/while`, nearest payloadless `break/continue`, bounded ownership fixed points, and exact per-edge/per-iteration cleanup.
- Pure typed loop invariants and `decreases`, concrete `diverge` backedge policy, deterministic unresolved entry/preservation/nonnegative/strict-decrease templates, owning-IR retention, and runtime erasure; no termination proof is implied.
- Statement-form `panic Text`, pure proof-backed `unreachable`, and `require Bool else Never`, with explicit IR, isolated ownership flow, deterministic cleanup, unresolved obligations, and structured interpreter failures.
- Structural tuple expressions, types, and recursive patterns of arity 2 through 16; static numeric projection places; recursive equality, Copy/affine ownership, partial moves, authority provenance, owning IR, interpreter values, and inspection-v3 projections.
- One canonical three-module executable-core conformance application that passes formatter check, compilation, authored tests, effect inspection, inspection-v3 projection, and hosted execution while jointly exercising imports, stable identity, data types, generics/traits, recursive matching, ownership/mutation/cleanup, typed errors, configured runtime failure/unwind, contracts/refinements, and standard capabilities.

12.2.1 Current Production Internals: P1 through P2.6

IMPLEMENTED These separate unstable owners cover the frozen E6 closure, not every form executable by the reference interpreter. They are compiler-internal, not the production compilation session/CLI, public serialization, a runtime ABI, or a backend. All builds are bounded and transactional, with independent validation and buffered deterministic rendering.

Complete	Current boundary
P1	Callable-scoped target-neutral CFG MIR for every bodyful E6 callable: SSA/block parameters, explicit storage and affine moves/borrows, projected/partial-move paths, calls and normal-only writeback, loops, propagation, constructors/refinements, matches/guards, contracts, exact handlers, generic/effect/evidence metadata, and cleanup on every exit. Independent dominance/SSA and affine-value validation, canonical internal rendering, and a bounded evaluator/trace are present.
P2.1	A symbolic program owner discovers deterministic entry/test/fixture closures, caches bodyful MIR templates, and records approved bodyless capability imports, specialization demands, and callable/provider references. It does not itself specialize or choose layout.
P2.2	Canonical monomorphic plans intern callable instances, concrete types/effects, receiver/type arguments, dispatch evidence, typed imports, and complete substitutions. Identical recursion deduplicates; expanding generic recursion and exhausted budgets reject transactionally.
P2.3	Concrete materialization owns specialized signatures, types, locals, places, values, blocks, terminators, edges, effects, bindings, and handler frames. Executable dispatch has no generic parameters, effect tails, or forwarded evidence; symbolic clones remain authentication-only. Concrete SSA, affine dataflow, cleanup, contracts, and closure are independently revalidated.

Complete	Current boundary
P2.4	Canonical target-neutral recipes cover primitives, aggregates/sums, distinct/refined wrappers, functions, and capabilities; source-order fields/variants, explicit semantic tags, inhabited/zero-size/Copy/drop classifications, and exact callable producers. Reachable open enums reject. This owner selects no byte layout or ABI.
P2.5	Explicit target descriptors select checked sizes, alignment, padding, source-order field offsets, u32 sum tags/payloads, projection maps, and text/callable/capability objects. Pointer widths 4/8 are supported; the initial Wasm32 descriptor is little-endian. Nonzero inhabited aggregates use uniform indirect storage; wrappers follow backing layout. Independent layout validation rejects overflow, incomplete reconstruction, and borrowed aliases.
P2.6	Representation-aware plans close accesses, construction and capability inheritance, recursive pattern tests/extractions, Option/Result propagation, checked arithmetic/equality, snapshots, callable producers, and handler bindings. Synthetic predicate CFGs and approved import-contract envelopes complete the source-independent executable boundary; source IDs are authenticated provenance only. Loop proof obligations remain runtime-erased.

12.2.2 Predicate Bodies and Finite Closure

P2.6b2 completes rich immutable predicate CFGs, replacing the earlier scalar-only checkpoint. Bodies support scalar/Text/Unit constants, direct contextual inputs, checked operations, short-circuit and conditional control, exact calls and function/bound-operation values under purity and finite-closure rules, aggregate/tuple/sum/wrapper/refined construction, recursive matches/guards/bindings, immutable blocks/locals, and nested refinement body references. There is no `UNRESOLVED_BODY` state. Instance and approved import contexts share this bounded vocabulary; import-owned requires/snapshots/ensures use envelope-local snapshot slots, not fake callable CFG coordinates. Contract/guard propagation remains rejected by `SOL-CONTRACT-002`; malformed propagation records are validated and rejected, not a newly lowerable source form.

Ordinary and predicate callback sites require a direct static definition or bound-operation callee. Current P1 cannot materialize ordinary first-class static callback forms, so ordinary body callbacks and dynamic, conditional, or aliased producers reject at the program boundary. Static callable-value references in pure predicates are supported within the finite plan. Contextless generic, receiver, and effect-polymorphic roots reject; trait evidence must come from compatible concrete incoming contexts. Reachable bodyful capability closures are not made production-lowerable merely because the interpreter executes them.

Callable MIR contract envelopes remain bounded: contracted generic/effect-parameter and exclusive forms, plus fallible/projected snapshots, reject transactionally. Supported snapshots are direct unprojected `Int64`, `Bool`, or Unit locals. Approved bodyless import contracts are separately closed by P2.6 and are not globally unsupported. The independent P1 evaluator still uses a bounded pure source-obligation evaluator; it is not the P2.6 source-independent operation owner. Its MIR-vs-owning-IR comparisons cover results/failure spans, host behavior, and cleanup, not future interpreter/Wasm conformance.

The lifetime chain is operations -> layout -> representation -> materialization -> plan -> program -> owning IR. Borrowed upstream owners must remain alive and immutable. Validators authenticate executable fields, CFG/SSA/type/ownership relations, provenance, and resource censuses; trusted mutable C owners are not arbitrary-pointer recovery APIs or public formats.

Baseline evidence includes all 14 bodyful E6 callables and four approved imports in the all-roots closure. The layout census has 21 types, 11 fields, nine variants, and five projections. E6's four single-block predicate bodies describe that fixture, not a restriction on P2.6b2 or a count of the compiler suite. E6 conformance is not usability evidence or proof of maintenance advantage; no compiler tests were rerun for this manual refresh.

12.2.3 Remaining Boundary

Symbols/linkage (P2.7), whole-program freeze/census (P2.8), production runtime ABI, backend cleanup policy, Wasm emission, reproducible `sol build`, and interpreter/Wasm differential execution remain open. Full public IR, semantic patch tooling, logical call-site substitution/normalization, SMT discharge, refinement projection/reasoning, and unresolved loop/decreases/unreachable proof obligations remain outside the delivered boundary. Broader traits, closures, resources, lifetime/reference values, collections, unsafe/FFI, manifests/dependencies, member/local IDs, concurrency, general handlers, and the rest of the feature vision remain separately gated target work.

12.3 Phased Roadmap

This snapshot mirrors the live ledger, not a second queue. Completion of P2.1-P2.6 does not complete P2.

Phase	Scope and exit criteria
E1-E6 / complete	Bounded shared compilation, hardened input/host boundaries, explicit entry-point, trusted interpreter profiles, <code>sol run</code> , runtime contracts/refinements, and E6 conformance.
P1 / complete	Frozen-E6 callable CFG MIR, dominance/affine validation, rendering, and bounded reference differential evaluation.
P2 / open	P2.1-P2.6 complete. P2.7 symbols/linkage is next overall and production; then M1; then P2.8 complete concrete-program freeze, rendering, and census. No runtime ABI or Wasm emission at this exit.
P3 / open	Target-independent call/result/failure ABI, allocation and owned operations, panic/cleanup, exact trusted hosting and handlers, then validated runtime-lowered closure. P3.W1 is a gated timeboxed integration experiment.
P4 / open	Select/pin an established Wasm toolchain; emit the frozen represented CFG; implement checks, cleanup, exact host adapter, and E6 Wasm execution. No implicit Component Model scope.
P5 / open	Immutable build API/artifacts, deterministic <code>sol build</code> writes, artifact execution, interpreter/Wasm differential tests, and byte-identical release/reproducibility acceptance.
M1-M5 / proposed	Workload charter, bounded context packets, conservative deltas, ordinary-edit validation/approval, and protected held-out evaluation. Optional M4E is only a basic editor slice. No M feature is implemented.
Deferred breadth	Proof/SMT, schemas, full public IR and patch language, richer libraries/numerics, resources, concurrency/workflows, general handlers, reflection, real-time profiles, native/VM targets, and ecosystem/stabilization work require explicit workload gates.

12.4 Minimum Viable Sol

Distinguish the minimum viable **experiment** from the long-term full-feature target. A small test of the thesis can use the existing language, interpreter, diagnostics, and selected projections to support ordinary source edits and protected review. It does not wait for full public IR, SMT, a backend, or a patch DSL. M1 first determines whether the projections suffice and records unknowns or blockers rather than silently activating those larger systems.

Proposal	Gate and evidence
M1	After P2.7, select one capability-restricted hosted business-logic/validation workload, user, host boundary, protected baseline, held-out tasks/tests, timebox, metrics, stop/go thresholds, projection gaps, and subsequent interleaving. If infeasible, record the blocker and continue P2.8.
M2	After M1 feasibility and P2.8, build declaration-centered packets from source, signatures, effects, contracts, and available references. Record snapshot hash, compiler/schema/options/selection metadata and omissions; unavailable is not empty. Task 58 slice only.
M3	After M2, compare checked snapshots conservatively, validate identities/base hashes, reject stale or ambiguous comparisons, and expose body edits and unknowns. A changed predicate does not establish weakening. Task 57 slice only.
M4	After M2/M3, exercise ordinary edit/check/test/delta/human approval with protected tests and policy. Contract or authority changes require explicit approval; no silent repair weakening. Only task 56's validation/approval slice, not patch syntax or proof of correctness.
M4E	Optional timeboxed diagnostics/navigation/packet presentation after M1/M2 projection feasibility; not full LSP and not an M5 prerequisite. Task 55 slice only.

Proposal	Gate and evidence
M5	After M4, compare against source-only work on protected held-out tasks with equal compiler/test access and matched model/budgets where relevant. Measure correctness, regressions, review time, context, iterations, diagnostic usefulness, and false confidence. Report negative/inconclusive results and decide continue, narrow, or stop.

After P2.8 these experiments may run independently alongside P3, but their internal dependencies still apply. No charter or tool is delivered by this refresh. Recoverable refinement validation/projection, small data facilities, test ergonomics, editor/latency/installation improvements, and finer effect families require separately bounded decisions driven by workload evidence.

P3.W1 is timeboxed after P3.1 has tested call/result/failure conventions: record candidate tool versions and validate/execute a minimal scalar module to expose ABI/tool mismatches. Allocation, cleanup, or host imports enter only after relevant P3.2-P3.4 conventions are tested. This informs, but does not complete, toolchain selection, P4 backend integration, component support, E6 Wasm execution, or build tooling.

The fuller target still includes stable semantic graph interfaces, intent metadata, semantic patches/change reports, progressive proof, and mature deployment tooling. Larger workflows/transactions should first gain library/host-provider experience; broad numerics/units, general resumptive handlers, reflection, and real-time certification remain proposed. None is required merely to test whether bounded semantic context helps maintenance.

12.5 Major Risks

Risk	Mitigation
Unmeasured product thesis	Use M1/M5 workload and held-out maintenance evidence; backend success and E6 conformance do not establish user benefit.
False confidence	Distinguish declared, runtime-checked, proved-under-assumptions, and unknown; never infer behavior preservation from unchanged metadata.
Feature interaction complexity	Small formal core; every surface feature lowers into it; RFC interaction matrices.
Verification brittleness	Modular contracts, stable IDs, proof caching, deterministic budgets, proof-health metrics.
Ownership ergonomics	Explainable region inference, persistent collections, scoped mutation, targeted sharing.
Effect annotation fatigue	Infer locally, normalize publicly, aliases, meaningful authority granularity.
Slow builds	Separate check/verify, semantic hashes, isolated parallel solvers, lightweight analysis first.
Model-specific tooling	Stable generic protocols usable by IDEs, linters, CI, and multiple AI systems.
Ecosystem cold start	Excellent C/Wasm interop, generated bindings, import tools, narrow high-value domain.
Unsound escape hatches	Small unsafe core, assumption ledgers, audited primitives, transformation verification.

12.6 Open Design Questions

Some v0.1 questions now have partial executable decisions:

- **Handlers:** 1.0 policy is still open, but the bootstrap has chosen exact capability-backed singleton-target handlers as a safe experimental subset. Broader resumptive/general handlers await ownership interaction work.
- **Mixed authority:** representation is decided for the bootstrap as normalized finite lexical may-origin sets with conservative expansion; runtime dynamic authority matching and long-term IR/ABI representation remain open.
- **Contracts:** syntax, signature-scope resolution, **Bool** typing, purity, **Result**-only outcomes, snapshots, deterministic templates, reference CHECK/IGNORE policy, and always-checked direct refined construction are implemented. Recoverable validation/projection APIs, proof-directed policy, logical IR, and proof discharge remain open.
- **Recursive effects:** the bootstrap decision is least-fixed-point SCC inference for omitted private closed rows. The target design of row variables and polymorphic recursive effects remains open.

Genuinely open target-design questions are:

1. How much dependent typing belongs outside ghost/proof code: compile-time naturals/protocol indices only, or a broader value-dependent fragment?
2. Is asynchronous suspension a `task.suspend` effect or a computation kind containing other effects?
3. Can protocol-state erasure and dynamic dispatch remain ergonomic without hiding runtime failure?
4. Which proof language best balances readable Sol syntax with established theorem-proving infrastructure?
5. Should native packages standardize an ABI early or rely on C/Wasm while the object model stabilizes?
6. How do semantic IDs evolve across forks, vendoring, and generated code?
7. Which resource-cost claims are portable and which require target certification?
8. Can registries enforce transitive capability policy without making legitimate platform abstractions unusable?
9. What evidence permits an AI repair to be labeled safe, behavior-preserving, or proof-preserving?
10. Should general handlers enter 1.0 after ownership is proven, or remain a later extension beyond exact capability-backed handlers?
11. Should named arguments retain formal-parameter evaluation order long term? The current interpreter executes in canonical formal order, not written operand order; reordering a signature can change effects even when named call syntax is unchanged. Alternatives need an explicit decision and tests, not a documentation change.
12. Is mandatory global declaration sorting worth lost locality and review churn? The current formatter preserves source order.

12.7 Success Criteria

Sol's maintenance thesis succeeds only if it measurably reduces defects or review effort without weakening constraints. Evaluation includes correctness, regressions, effect-leak prevention, review effort, onboarding, edit-loop latency, and context required for a correct change, not only speed or line count. E6 conformance, compiler regression tests, application usefulness, and maintenance outcomes are separate evidence categories. A negative or inconclusive M5 result is useful evidence, not a reason to move the success criteria.

13 Worked Reference Examples

These examples revise the v0.1 reference corpus. Unless explicitly stated, they express target design and are not bootstrap conformance tests, executable demonstrations, or formal proofs. Provider APIs, domain types, collections, serialization, transaction state views, resource policies, and host wiring must be designed separately. Effect names do not grant ambient authority: snippet-local resources must be supplied by explicit capabilities or an enclosing provider scope.

13.1 HTTP Handler with Explicit Authority

```
module api.accounts
use http.{Request, Response, Status}
use identity.AccountId
public function get_account(
  request: Request,
  repository: capability AccountRepository,
  authorization: capability Authorization,
  tracing: capability TraceSink,
) -> Result<Response, HandlerError>
effects {
  database.read<repository>
  authorization.check<authorization>
  tracing.emit<tracing>
} {
  let account_id = request.path.parameter("account_id")
    .parse<AccountId>().map_error(HandlerError.invalid_account_id)?
  authorization.require(
    request.principal, Permission.read_account(account_id),
  )?
  let account = repository.find(account_id)?
  return match account {
    some(value) => Response.json(Status.ok, value.public_view())
    none => Response.empty(Status.not_found)
  }
}
```

Listing 49: Service handler without ambient repository or authorization access.

The signature exposes operational authority; target tests can replace the providers. Permission to invoke a capability does not itself establish business authorization, tenant isolation, or information-flow safety. Those require host policy, domain checks, framing, and explicitly scoped evidence. A cache wrapper could require evidence that authorization precedes protected cached data, not infer it from an effect row.

13.2 Idempotent Inventory Reservation

TARGET DESIGN Formal transaction pseudocode, not executable bootstrap Sol or a proof. The provider must supply serializable isolation, rollback, entry-state views, and commit-coupled events. State projections below are logical views of the explicitly supplied database, not ambient mutable globals or effectful bootstrap contract reads. The retry branch covers only the same order/item/quantity; conflicting-quantity retry policy remains open.

```

intent reserve_inventory {
  preserve:
    - available stock never becomes negative
    - retries do not reserve twice
    - stock update and reservation record commit atomically
}
public transaction reserve(
  database: capability InventoryDb, clock: capability Clock,
  order: OrderId, item: ItemId, quantity: Quantity,
) -> Result<Reservation, ReservationError>
isolation = serializable
effects { database.transaction<database> clock.read<clock> }
// inventory/reservations are this provider's logical transaction views.
// Domain: quantity > 0, nonnegative stock, consistent reservation keys;
// an existing order/item has the same quantity.
ensures {
  success => reservations.contains(order, item)
  success => result == reservations[order, item]
  success => if old(reservations.contains(order, item)) {
    result == old(reservations[order, item])
    and inventory[item].available == old(inventory[item].available)
  } else {
    result.order == order and result.item == item
    and result.quantity == quantity
    and inventory[item].available
      == old(inventory[item].available) - quantity
  }
  failure => inventory == old(inventory)
  failure => reservations == old(reservations)
  preserves unrelated stock and reservation entries
} {
  match reservations.find(order, item) {
    some(existing) => return existing
    none => proceed with creation below
  }
  let stock = inventory.lock(item)?
  require stock.available >= quantity else {
    return ReservationError.insufficient_stock(
      requested = quantity, available = stock.available,
    )
  }
  stock.available -= quantity
  let reservation = Reservation {
    order, item, quantity, created_at = clock.now(),
  }
  reservations.insert(reservation)
  emit commit InventoryReserved(reservation)
  return reservation
}

```

Listing 50: Target reservation: creation consumes stock; a same-request replay preserves it.

Here `old(...)` denotes a logical pre-state observation; the guarded reservation lookup is defined only when the pre-state contains that key. It is not the bootstrap's eager per-occurrence snapshot evaluation. For a new successful reservation with stock 10 and quantity 3, stock becomes 7 and one matching record/event is committed. Replaying the same request returns that record with stock still 7 and no new event. Failure rolls back both state views. The provider's atomicity and state framing are assumptions awaiting design and validation, not proved properties.

Required target tests cover creation, same-quantity retry, insufficient stock/rollback, unrelated-state preservation, and concurrent duplicate requests under the chosen isolation policy. A retry with a different quantity is deliberately outside this example's domain until an explicit conflict policy and tests are approved; no shipped behavior is invented here.

13.3 Embedded Controller

```

record ControllerState { integral: Float32, previous_error: Float32 }
function control_tick(
  state: inout ControllerState,
  target: Celsius,
  measured: Celsius,
  dt: Second,
) -> DutyCycle
effects { pure }
requires { 0 s < dt <= 100 ms }
ensures { 0 percent <= result <= 100 percent }
resources {
  profile = real_time
  stack <= 512 B
  heap == 0 B
  worst_case_time <= 50 us
} {
  let error = target - measured
  state.integral = clamp(
    state.integral + error.value * dt.value, -1000.0, 1000.0,
  )
  let derivative = (error.value - state.previous_error) / dt.value
  state.previous_error = error.value
  return DutyCycle.clamped(
    kp * error.value + ki * state.integral + kd * derivative,
  )
}

```

Listing 51: Heap-free real-time function with units and bounded result.

13.4 Protocol-Typed Database Session

```

protocol Session {
  state disconnected
  state connected(connection: DbConnection)
  state transaction_open(connection: DbConnection, tx: Tx)
  transition connect(from: disconnected, config: DbConfig)
    -> Result<connected, ConnectError>
  transition begin(from: connected)
    -> Result<transaction_open, BeginError>
  transition commit(from: transaction_open)
    -> Result<connected, CommitError>
  transition rollback(from: transaction_open) -> connected
  transition close(from: connected) -> disconnected
}
using session = Session.connect(config)?
using tx = session.begin()?
repository.update(tx, record)?
session = tx.commit()?

```

Listing 52: Legal lifecycle encoded in resource state.

13.5 Semantic Refactor

```
patch package inventory-service {  
  select calls to legacy_clock.now  
  replace with capability clock.now  
  add capability parameter clock: capability Clock  
    to nearest public entrypoint  
  thread parameter through private callers  
  preserve {  
    returned timestamps  
    error variants  
    allocation behavior  
  }  
  require {  
    no remaining effect environment.read<TZ>  
    all tests use explicit clock handlers  
  }  
}
```

Listing 53: Cross-cutting refactor expressed by meaning, not search.

Appendix A. Syntax and Grammar Sketch

This appendix is illustrative, not parser-complete normative grammar. The authoritative edition grammar should be generated from compiler source. The sketch establishes regularity and reveals syntax interactions. It is updated with executable-core handler and authority forms.

```

source_file := module_decl edition_decl? use_decl* declaration*
module_decl := "module" module_path NEWLINE
edition_decl := "edition" INTEGER NEWLINE
use_decl := "use" use_path NEWLINE
declaration := visibility? annotation* (
    record_decl | enum_decl | type_decl | trait_decl | implementation_decl
    | capability_decl
    | function_decl | protocol_decl | transaction_decl | workflow_decl
    | intent_decl | spec_decl | test_decl
)
function_decl := "function" qualified_name function_generic_params?
    "(" parameter_list? ")" "->" type
    authority_clause? effect_clause? requires_clause? ensures_clause?
    cost_clause? resource_clause? block
effect_clause := "effects" "{" effect_entry* "}"
type_generic_params := "<" TYPE_NAME ("," TYPE_NAME)* ">"
function_generic_params := "<" TYPE_NAME (":" TYPE_NAME)?
    ("," TYPE_NAME (":" TYPE_NAME)?)*
    ("," "effects" TYPE_NAME)? ">" | "<" "effects" TYPE_NAME ">"
authority_clause := "authority" "{" "result" "derives_from"
    (IDENTIFIER | "Self") "}"
requires_clause := "requires" "{" contract_condition* "}"
ensures_clause := "ensures" "{" contract_condition* "}"
contract_condition := outcome? expression
outcome := ("success" | "failure") "=>"
record_decl := "record" TYPE_NAME type_generic_params? version_clause? "{" field_decl* "}"
enum_decl := "open"? "enum" TYPE_NAME type_generic_params? "{" variant_decl* "}"
trait_decl := "trait" TYPE_NAME "{" trait_method* "}"
implementation_decl := "implementation" TYPE_NAME "for" type
    "{" implementation_method* "}"
trait_method := "function" IDENTIFIER "(" parameter_list? ")" "->" type
    effect_clause
implementation_method := "function" IDENTIFIER
    "(" parameter_list? ")" "->" type effect_clause block
capability_decl := "capability" TYPE_NAME
    ("derives_from" IDENTIFIER ":" "capability" TYPE_NAME)?
    "{" capability_member* "}"
type_decl := "type" TYPE_NAME type_generic_params? "="
    ("distinct" type | "refined" type refinement)
statement := let_stmt | var_stmt | assignment | return_stmt
    | panic_stmt | unreachable_stmt | require_stmt
    | emit_stmt | using_stmt | expression_stmt
panic_stmt := "panic" expression
unreachable_stmt := "unreachable" "because" "{" expression "}"
require_stmt := "require" expression "else" block
expression := literal | path | call | block | if_expr | match_expr
    | binary_expr | unary_expr | record_expr | type_apply_expr
    | lambda_expr | handle_expr
type_apply_expr := (path | field_expr) "<" type_list ">"
    # followed by "(" or "." or "{"
handle_expr := "handle" effect_name "<" expression ">"
    "with" expression block
match_expr := "match" expression "{" match_arm* "}"
match_arm := pattern guard? "=>" expression
guard := "if" expression
pattern := " " | "true" | "false" | IDENTIFIER
    | IDENTIFIER "(" pattern_list? ")"

```

```

    | "(" pattern "," pattern "(" pattern)* ","? ")"
    | TYPE_NAME "{" record_pattern_fields? "}"
pattern_list := pattern "(" pattern)* ","?
record_pattern_fields := record_pattern_field
    "(" record_pattern_field)* ","?
record_pattern_field := IDENTIFIER ("=" pattern)?
result_type := "Result" "<" type "," type ">"
option_type := "Option" "<" type ">"
function_type := "function" "(" type_list? ")" "->" type
    (effect_clause | "effects" TYPE_NAME)

```

Listing 54: Condensed grammar sketch.

In canonical bootstrap source, **authority** precedes **effects**, followed by **requires** and **ensures**; each clause occurs at most once. Newlines or commas separate contract conditions. **success** and **failure** are valid only in **ensures** on declarations returning **Result**<T, E>; **result** and **old** are contextual contract forms, and **old** is valid only in postconditions. A free-function authority source names a direct capability parameter, while a capability member uses **Self**. The bounded bootstrap permits one inline trait bound per free-function type parameter and one trailing **effects** E parameter; callback types use **effects** E, while the declaration row names E inside braces. Trait and implementation methods begin with **self: Self**, require explicit closed effects, and do not admit authority or contract clauses. Exact handlers always have an explicitly parameterized target and cannot transform an unresolved row. In expressions, a balanced <...> is recognized as type arguments only on a path/field-like head when it closes before (, ., or {; ordinary comparisons remain binary expressions. A zero-arm match is accepted semantically only for an empty finite/uninhabited closed enum and has type **Never**. Bare payloadless variants are accepted only as top-level arm patterns; nested payloadless variants require **variant()**.

Appendix B. Standard Effect Taxonomy

Concrete effects are parameterized by capabilities, resources, or roots. Aliases may group domain effects but expand to canonical rows in public semantic IR. In the bootstrap, declaration rows are closed except while symbolically checking the bounded callback-driven row parameter; every concrete call instantiation closes that row. Parameter- and Self-dependent atoms expand over normalized may-origin roots but cannot be captured by a row argument.

Effect	Meaning and notes
<code>pure</code>	An empty row alone does not establish totality, absence of runtime failure, determinism, or proof eligibility; predicates have additional restrictions.
<code>diverge</code>	May not terminate; excluded from proofs/compile-time evaluation.
<code>panic</code>	Abnormal termination due to violated invariant.
<code>memory.allocate</code>	Allocation through a specified allocator/region.
<code>memory.pin</code>	Address stability constraining movement/collection.
<code>clock.read</code>	Wall, civil, or monotonic clock capability.
<code>random.pseudo</code>	Deterministic pseudo-random state.
<code>random.secure</code>	Secure host entropy capability.
<code>filesystem.read/write</code>	Filesystem-root-constrained access.
<code>network.call/listen</code>	Named outbound service/inbound endpoint class.
<code>database.read/write/transaction</code>	Repository/database capability operations.
<code>console.read/write</code>	Interactive terminal or stream.
<code>environment.read</code>	Host environment; discouraged outside wiring.
<code>process.spawn</code>	Host process creation/control.
<code>task.spawn/suspend</code>	Child creation or yielding/suspension.
<code>sync.lock</code>	Synchronization acquisition.
<code>channel.send/receive</code>	Typed endpoint use.
<code>tracing.emit</code>	Structured observability event.
<code>unsafe</code>	Manually established memory/type invariants.
<code>ffi.call</code>	Foreign ABI boundary; safe generated wrappers may encapsulate it.

Appendix C. Diagnostic and Patch Schemas

C.1 Diagnostic Envelope

```
{
  "schema": "sol.diagnostic/1",
  "code": "SOL-EFFECT-014",
  "severity": "error",
  "message": "undeclared effect network.call<AuditService>",
  "subject": {
    "stable_id": "billing.transfer.execute.v1",
    "kind": "function"
  },
  "locations": [{
    "file": "billing/transfer.sol",
    "start": { "line": 42, "column": 9 },
    "end": { "line": 42, "column": 31 },
    "role": "primary"
  }],
  "facts": {
    "attempted_effect": "network.call<AuditService>",
    "declared_effects": ["database.transaction<Accounts>", "clock.read"]
  },
  "repairs": [
    {
      "kind": "add_effect",
      "safety": "behavior_expanding",
      "requires_approval": "capability_policy"
    },
    { "kind": "inject_capability", "safety": "architecture_change" }
  ]
}
```

Listing 55: Proposed diagnostic JSON envelope.

C.2 Patch Envelope

```
{
  "schema": "sol.semantic-patch/1",
  "base_graph": "sha256:...",
  "operations": [{
    "op": "add_parameter",
    "target": "billing.transfer.execute.v1",
    "after": "amount",
    "parameter": { "name": "memo", "type": "Option<Text>" }
  }],
  "preserve": ["public_errors", "atomicity", "existing_success_behavior"],
  "author": {
    "kind": "tool",
    "name": "assistant",
    "signature": "optional-attestation"
  }
}
```

Listing 56: Proposed semantic patch exchange format.

Appendix D. Comparative Positioning

Sol is intentionally synthetic. No current language provides the complete combination proposed here. This comparison describes influence, not direct compatibility.

System	Relevant strengths	What Sol adds or changes
F* + Pulse	Dependent/refinement types, effects, SMT, ghost erasure, mutable-state/concurrency proofs.	Conventional systems surface, semantic editing, progressive modes, change reports, unified toolchain.
Dafny	Contracts, ghost code, automated verification, familiar imperative syntax.	Ownership, general effects, native execution, capabilities, patch protocols.
Rust	Ownership, borrowing, enums, exhaustive matching, typed errors, systems ecosystem.	Native effects/contracts, proofs, protocols, schemas, semantic interfaces.
Koka	Precise effect inference, algebraic handlers, effect polymorphism.	Ownership, nominal systems types, contracts, transactions, schemas, deployment profiles.
Idris 2	Dependent/quantitative types, linear resources, protocol examples.	Runtime contracts with proposed progressive SMT proof, effects, maintenance tooling, less-dependent everyday language.
SPARK	Contracts, type predicates, absence-of-runtime-error proof, assurance discipline.	ADTs, ownership ergonomics, effect/capability rows, modern concurrency, agent tooling.
Pony	Actors and reference capabilities for race-safe sharing.	General ownership, proofs, typed errors, transactions, schemas, broader backends.
MLIR	Multilevel IR, textual/in-memory/serialized forms, reusable transforms.	Language-specific stable graph with contracts, effects, ownership, and patch identity.

D.1 Closest Current Language

For the complete conceptual design, F* with Pulse is closest because it combines dependent/refinement types, effectful programming, automated proof, ghost erasure, mutable state, and concurrent separation logic. Dafny is closest to the readable contract surface; Rust to production implementation substrate; Koka to target effect rows and broader handler model.

Sol's differentiating claim is integration of safe implementation, explicit authority, progressive proof, stable semantic representation, and change-oriented tooling. The bootstrap does not yet realize that complete comparison; it tests a narrow executable core.

Appendix E. References and Design Influences

These primary sources informed comparison and implementation discussion; they are not normative dependencies.

- [R1] F* - A Proof-Oriented Programming Language: dependent typing, effects, SMT/tactics.
- [R2] F* Tutorial, Computational Effects: computation types, user effects, divergence, ghost computation.
- [R3] Pulse: proof-oriented mutable state, concurrency, specifications, and separation logic.
- [R4] Dafny Documentation and Reference Manual: contracts, framing, static verification, ghost features.
- [R5] The Koka Programming Language: effect inference, row polymorphism, handlers, pure/effectful types.
- [R6] The Rust Programming Language, Understanding Ownership: ownership and borrowing without GC.
- [R7] Idris 2 Documentation, Linear Resources: quantitative resources and state-indexed protocols.
- [R8] SPARK User's Guide, Subprogram and Type Contracts: high-assurance contract checking.
- [R9] Pony Tutorial, Reference Capabilities: concurrency-safe sharing and transfer.
- [R10] MLIR Language Reference and Rationale: multilevel textual/in-memory/serialized IR.
- [R11] Cranelift: code generator and CLIF IR for development and JIT-style execution.
- [R12] WebAssembly Component Model: composable components and rich cross-language interfaces.

Closing Statement

Sol is a proposal for a new interface between intent, implementation, verification, and change. Its syntax is conservative. Its novelty is making effects, authority, contracts, resource behavior, semantic identity, and change consequences normal programming rather than layers reconstructed afterward.

The most important experiment is not whether Sol compiles a benchmark. It is whether a human or model can modify a nontrivial system with less hidden context, receive bounded meaningful obligations, and demonstrate mechanically or operationally that requested behavior changed while unrelated behavior did not.

NEXT WORK Follow the sole live queue: P2.7 -> M1 -> P2.8. M1 selects a workload and publishes interleaving; the bounded experiment does not require full IR, SMT, or patch syntax. Consult the implementation snapshot for compatibility boundaries and the analysis for findings and evaluation rationale. Future RFCs should separate target semantics, implemented behavior, and unresolved decisions without creating competing priorities.

End of Design Specification v0.2